WWW.SabzElco.IR

4 Delay

4.1 Introduction

In Chapter 1 we learned how to make chips that work. Now we move on to making chips that work *well*. The two most common metrics for a good chip are speed and power, discussed in this chapter and Chapter 5, respectively. Delay and power are influenced as much by the wires as by the transistors, so Chapter 6 delves into interconnect analysis and design. A chip is of no value if it cannot reliably accomplish its function, so Chapter 7 examines how we achieve robustness in designs.

The most obvious way to characterize a circuit is through simulation, and that will be the topic of Chapter 8. Unfortunately, simulations only inform us how a particular circuit behaves, not how to change the circuit to make it better. There are far too many degrees of freedom in chip design to explore each promising path through simulation (although some may try). Moreover, if we don't know approximately what the result of the simulation should be, we are unlikely to catch the inevitable bugs in our simulation model. Mediocre engineers rely entirely on computer tools, but outstanding engineers develop their physical intuition to rapidly predict the behavior of circuits. In this chapter and the next two, we are primarily concerned with the development of simple models that will assist us in understanding system performance.

4.1.1 Definitions

We begin with a few definitions illustrated in Figure 4.1:

- *Propagation delay time*, t_{pd} = maximum time from the input crossing 50% to the output crossing 50%
- *Contamination delay time*, t_{cd} = minimum time from the input crossing 50% to the output crossing 50%
- *Rise time*, t_r = time for a waveform to rise from 20% to 80% of its steady-state value
- *Fall time*, t_f = time for a waveform to fall from 80% to 20% of its steady-state value
- *Edge rate*, $t_{rf} = (t_r + t_f)/2$

Intuitively, we know that when an input changes, the output will retain its old value for at least the contamination delay and take on its new value in at most the propagation delay. We sometimes

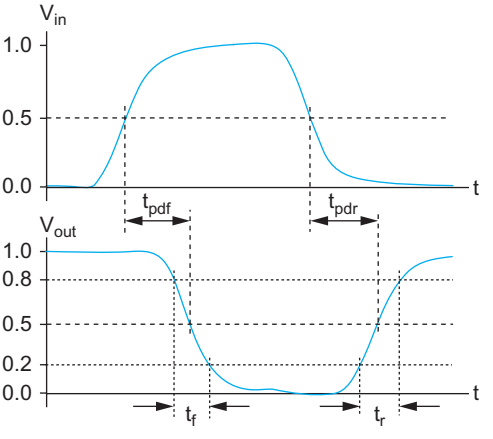


FIGURE 4.1 Propagation delay and rise/fall times

differentiate between the delays for the output rising, t_{pdr}/t_{cdr}, and the output falling, t_{pdf}/t_{cdf}. Rise/fall times are also sometimes called *slopes* or *edge rates*. Propagation and contamination delay times are also called *max-time* and *min-time*, respectively. The gate that charges or discharges a node is called the *driver* and the gates and wire being driven are called the *load*. Propagation delay is usually the most relevant value of interest, and is often simply called *delay*.

A *timing analyzer* computes the arrival times, i.e., the latest time at which each node in a block of logic will switch. The nodes are classified as inputs, outputs, and internal nodes. The user must specify the *arrival time* of inputs and the time data is required at the outputs. The arrival time a_i at internal node i depends on the propagation delay of the gate driving i and the arrival times of the inputs to the gate:

$$a_i = \max_{j \in fanin(i)} \{a_j\} + t_{pd_i} \tag{4.1}$$

The timing analyzer computes the arrival times at each node and checks that the outputs arrive by their required time. The *slack* is the difference between the required and arrival times. *Positive slack* means that the circuit meets timing. *Negative slack* means that the circuit is not fast enough. Figure 4.2 shows nodes annotated with arrival times. If the outputs are all required at 200 ps, the circuit has 60 ps of slack.

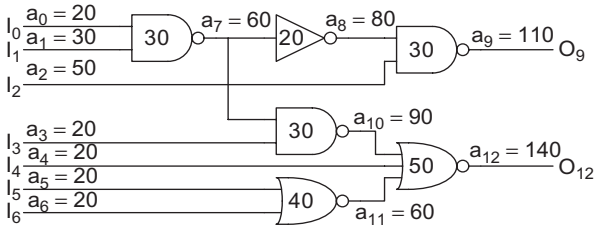


FIGURE 4.2 Arrival time example

A practical timing analyzer extends this arrival time model to account for a number of effects. Arrival times and propagation delays are defined separately for rising and falling transitions. The delay of a gate may be different from different inputs. Earliest arrival times can also be computed based on contamination delays. Considering all of these factors gives a window over which the gate may switch and allows the timing analyzer to verify that setup and hold times are satisfied at each register.

4.1.2 Timing Optimization

In most designs there will be many logic paths that do not require any conscious effort when it comes to speed. These paths are already fast enough for the timing goals of the system. However, there will be a number of *critical paths* that limit the operating speed of the system and require attention to timing details. The critical paths can be affected at four main levels:

- The architectural/microarchitectural level
- The logic level
- The circuit level
- The layout level

The most leverage is achieved with a good microarchitecture. This requires a broad knowledge of both the algorithms that implement the function and the technology being targeted, such as how many gate delays fit in a clock cycle, how quickly addition occurs, how fast memories are accessed, and how long signals take to propagate along a wire. Trade-offs at the microarchitectural level include the number of pipeline stages, the number of execution units (parallelism), and the size of memories.

The next level of timing optimization comes at the logic level. Trade-offs include types of functional blocks (e.g., ripple carry vs. lookahead adders), the number of stages of gates in the clock cycle, and the fan-in and fan-out of the gates. The transformation from function to gates and registers can be done by experience, by experimentation, or, most often, by logic synthesis. Remember, however, that no amount of skillful logic design can overcome a poor microarchitecture.

Once the logic has been selected, the delay can be tuned at the circuit level by choosing transistor sizes or using other styles of CMOS logic. Finally, delay is dependent on the layout. The floorplan (either manually or automatically generated) is of great importance because it determines the wire lengths that can dominate delay. Good cell layouts can also reduce parasitic capacitance.

Many RTL designers never venture below the microarchitectural level. A common design practice is to write RTL code, synthesize it (allowing the synthesizer to do the timing optimizations at the logic, circuit, and placement levels) and check if the results are fast enough. If they are not, the designer recodes the RTL with more parallelism or pipelining, or changes the algorithm and repeats until the timing constraints are satisfied. Timing analyzers are used to check *timing closure*, i.e., whether the circuit meets all of the timing constraints. Without an understanding of the lower levels of abstraction where the synthesizer is working, a designer may have a difficult time achieving timing closure on a challenging system.

This chapter focuses on the logic and circuit optimizations of selecting the number of stages of logic, the types of gates, and the transistor sizes. We begin by examining the transient response of an inverter. Using the device models from Chapter 2, we can write differential equations for voltage as a function of time to calculate delay. Unfortunately, these equations are too complicated to give much insight, yet too simple to give accurate results. This chapter focuses on developing simpler models that offer the designer more intuition. The RC delay model approximates a switching transistor with an effective resistance and provides a way to estimate delay using arithmetic rather than differential equations. The method of Logical Effort simplifies the model even further and is a powerful way to evaluate delay in circuits. The chapter ends with a discussion of other delay models used for timing analysis.

4.2 Transient Response

The most fundamental way to compute delay is to develop a physical model of the circuit of interest, write a differential equation describing the output voltage as a function of input voltage and time, and solve the equation. The solution of the differential equation is called the *transient response*, and the delay is the time when the output reaches $V_{DD}/2$.

The differential equation is based on charging or discharging of the capacitances in the circuit. The circuit takes time to switch because the capacitance cannot change its voltage instantaneously. If capacitance C is charged with a current I, the voltage on the capacitor varies as:

$$I = C\frac{dV}{dt} \tag{4.2}$$

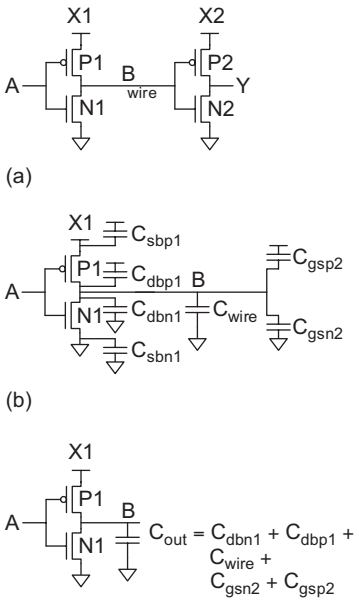


FIGURE 4.3 Capacitances for inverter delay calculations

Every real circuit has some capacitance. In an integrated circuit, it typically consists of the gate capacitance of the load along with the diffusion capacitance of the driver's own transistors, as discussed in Section 2.3. As will be explored further in Section 6.2.2, wires that connect transistors together often contribute the majority of the capacitance. The transistor current depends on the input (gate) and output (source/drain) voltages. To illustrate these points, consider computing the step response of an inverter.

Figure 4.3(a) shows an inverter $X1$ driving another inverter $X2$ at the end of a wire. Suppose a voltage step from 0 to V_{DD} is applied to node A and we wish to compute the propagation delay, t_{pdf}, through $X1$, i.e., the delay from the input step until node B crosses $V_{DD}/2$.

These capacitances are annotated on Figure 4.3(b). There are diffusion capacitances between the drain and body of each transistor and between the source and body of each transistor: C_{db} and C_{sb}. The gate capacitance C_{gs} of the transistors in $X2$ are part of the load. The wire capacitance is also part of the load. The gate capacitance of the transistors in $X1$ and the diffusion capacitance of the transistors in $X2$ do not matter because they do not connect to node B. The source-to-body capacitors C_{sbn1} and C_{sbp1} have both terminals tied to constant voltages and thus do not contribute to the switching capacitance. It is also irrelevant whether the second terminal of each capacitor connects to ground or power because both are constant supplies, so for the sake of simplicity, we can draw all of the capacitors as if they are connected to ground. Figure 4.3(c) shows the equivalent circuit diagram in which all the capacitances are lumped into a single C_{out}.

Before the voltage step is applied, $A = 0$. $N1$ is OFF, $P1$ is ON, and $B = V_{DD}$.

After the step, $A = 1$. $N1$ turns ON and $P1$ turns OFF and B drops toward 0. The rate of change of the voltage V_B at node B depends on the output capacitance and on the current through $N1$:

$$C_{\text{out}} \frac{dV_B}{dt} = -I_{dsn1} \tag{4.3}$$

Suppose the transistors obey the long-channel models. The current depends on whether $N1$ is in the linear or saturation regime. The gate is at V_{DD}, the source is at 0, and the drain is at V_B. Thus, $V_{gs} = V_{DD}$ and $V_{ds} = V_B$. Initially, $V_{ds} = V_{DD} > V_{gs} - V_t$, so $N1$ is in saturation. As V_B falls below $V_{DD} - V_t$, $N1$ enters the linear regime. Substituting EQ (2.10) and rearranging, we find the differential equation governing V_B.

$$\frac{dV_B}{dt} = -\frac{\beta}{C_{\text{out}}} \begin{cases} \dfrac{(V_{DD} - V_t)^2}{2} & V_B > V_{DD} - V_t \\ \left(V_{DD} - V_t - \dfrac{V_B}{2}\right) V_B & V_B < V_{DD} - V_t \end{cases} \tag{4.4}$$

During saturation, the current is constant and V_B drops linearly until it reaches $V_{DD} - V_t$. Thereafter, the differential equation becomes nonlinear. The response can be computed numerically. The rising output response is computed in an analogous fashion and is symmetric with the falling response if $\beta_p = \beta_n$.

Example 4.1

Plot the response of the inverter to a step input and determine the propagation delay. Assume that the nMOS transistor width is 1 μm and the output capacitance is 20 fF. Use the following long-channel model parameter values for a 65-nm process: $L = 50$ nm, $V_{DD} = 1.0$ V, $V_t = 0.3$ V, $t_{ox} = 10.5$ Å, $\mu = 80$ cm^2/V · s.

SOLUTION: The response is plotted in Figure 4.4. The input, A, rises at 10 ps. The solid blue line indicates the step response predicted by the long-channel model. The output, B, initially follows a straight line, as the saturated nMOS transistor behaves as a constant current source. B eventually curves as it approaches 0 and the nMOS transistor enters the linear regime. The propagation delay is 12.5 ps. The solid black line indicates the step response predicted by SPICE. The propagation delay is 15.8 ps, which is longer because the mobility used in the long-channel model didn't fully account for velocity saturation and mobility degradation effects. SPICE shows that B also initially rises momentarily before falling. This effect is called bootstrapping and will be discussed in Section 4.4.6.6. The dashed black line shows an RC model that approximates the nMOS transistor as a 1 kΩ resistor when it is ON. The propagation delay predicted by the RC model matches SPICE fairly well, although the fall time is overestimated. RC models will be explored further in Section 4.3.

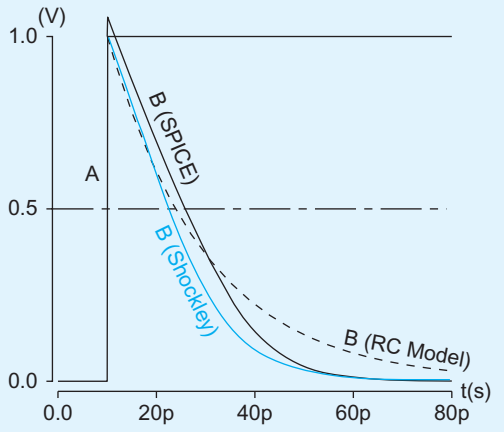


FIGURE 4.4
Inverter step response

In a real circuit, the input comes from another gate with a nonzero rise/fall time. This input can be approximated as a ramp with the same rise/fall time. Again, let us consider a rising ramp and a falling output and examine how the nonzero rise time affects the propagation delay.

Assuming $V_{tn} + |V_{tp}| < V_{DD}$, the ramp response includes three phases, as shown in Table 4.1. When A starts to rise, $N1$ remains OFF and B remains at V_{DD}. When A reaches V_{tn}, $N1$ turns ON. It fights $P1$ and starts to gradually pull B down toward an intermediate value predicted by the DC circuit response examined in Section 2.5. When A gets close enough to V_{DD}, $P1$ turns OFF and B falls to 0 unopposed. Thus, we can write the differential equations for V_B in each phase:

$$\begin{aligned} &\text{Phase 1} && V_B = V_{DD} \\ &\text{Phase 2} && \frac{dV_B}{dt} = \frac{I_{dsp1} - I_{dsn1}}{C_{out}} \\ &\text{Phase 3} && \frac{dV_B}{dt} = \frac{-I_{dsn1}}{C_{out}} \end{aligned} \tag{4.5}$$

TABLE 4.1 Phases of inverter ramp response

Phase	V_A	$N1$	$P1$	V_B
1	$0 < V_A < V_{tn}$	OFF	ON	V_{DD}
2	$V_{tn} < V_A < V_{DD} - \|V_{tp}\|$	ON	ON	Intermediate
3	$V_{DD} - \|V_{tp}\| < V_A < V_{DD}$	ON	OFF	Falling toward 0

The currents could be estimated using the long-channel model again, but working out the full model is tedious and offers little insight. The key observation is that the propagation delay increases because $N1$ is not fully ON right away and because it must fight $P1$ in Phase 2. Section 4.4.6.1 develops a model for how propagation delay increases with rise time.

More complex gates such as NANDs or NORs have transistors in series. Each series transistor sees a smaller V_{ds} and delivers less current. The current through the transistors can be found by solving the simultaneous nonlinear differential equations, which again is best done numerically. If the transistors have the same dimensions and the load is the same, the delay will increase with the number of series transistors.

This section has shown how to develop a physical model for a circuit, write the differential equation for the model, and solve the equation to compute delay. The physical modeling shows that the delay increases with the output capacitance and decreases with the driver current. The differential equations used the long-channel model for transistor current, which is inaccurate in modern processes. The equations are also too nonlinear to solve in closed form, so they have to be solved numerically and give little insight about delay. Circuit simulators automate this process using more accurate delay equations and give good predictions of delay, but offer even less insight. The rest of this chapter is devoted to developing simpler delay models that offer more insight and tolerable accuracy.

4.3 RC Delay Model

RC delay models approximate the nonlinear transistor I-V and C-V characteristics with an average resistance and capacitance over the switching range of the gate. This approximation works remarkably well for delay estimation despite its obvious limitations in predicting detailed analog behavior.

4.3.1 Effective Resistance

The RC delay model treats a transistor as a switch in series with a resistor. The *effective resistance* is the ratio of V_{ds} to I_{ds} averaged across the switching interval of interest.

A unit nMOS transistor is defined to have effective resistance R. The size of the unit transistor is arbitrary but conventionally refers to a transistor with minimum length and minimum contacted diffusion width (i.e., 4/2 λ). Alternatively, it may refer to the width of the nMOS transistor in a minimum-sized inverter in a standard cell library. An nMOS transistor of k times unit width has resistance R/k because it delivers k times as much current. A unit pMOS transistor has greater resistance, generally in the range of $2R$–$3R$, because of its lower mobility. Throughout this book we will use $2R$ for examples to keep arithmetic simple. R is typically on the order of 10 kΩ for a unit transistor. Sections 4.3.7 and 8.4.5 examine how to determine the effective resistance for transistors in a particular process.

According to the long-channel model, current decreases linearly with channel length and hence resistance is proportional to L. Moreover, the resistance of two transistors in series is the sum of the resistances of each transistor (see Exercise 2.2). However, if a transistor is fully velocity-saturated, current and resistance become independent of channel length. Real transistors operate somewhere between these two extremes. This also means that the resistance of transistors in series is somewhat lower than the sum of the resistances, because series transistors see smaller V_{ds} and are less velocity-saturated. The effect is more pronounced for nMOS transistors than pMOS because of the higher mobility and

greater degree of velocity saturation. The simplest approach is to neglect velocity-saturation for hand calculations, but recognize that series transistors will be somewhat faster than predicted.

4.3.2 Gate and Diffusion Capacitance

Each transistor also has gate and diffusion capacitance. We define C to be the gate capacitance of a unit transistor of either flavor. A transistor of k times unit width has capacitance kC. Diffusion capacitance depends on the size of the source/drain region. Using the approximations from Section 2.3.1, we assume the contacted source or drain of a unit transistor to also have capacitance of about C. Wider transistors have proportionally greater diffusion capacitance. Increasing channel length increases gate capacitance proportionally but does not affect diffusion capacitance.

Although capacitances have a nonlinear voltage dependence, we use a single average value. As discussed in Section 2.3.1, we roughly estimate C for a minimum length transistor to be 1 fF/μm of width. In a 65 nm process with a unit transistor being 0.1 μm wide, C is thus about 0.1 fF.

4.3.3 Equivalent RC Circuits

Figure 4.5 shows equivalent RC circuit models for nMOS and pMOS transistors of width k with contacted diffusion on both source and drain. The pMOS transistor has approximately twice the resistance of the nMOS transistor because holes have lower mobility than electrons. The pMOS capacitors are shown with V_{DD} as their second terminal because the n-well is usually tied high. However, the behavior of the capacitor from a delay perspective is independent of the second terminal voltage so long as it is constant. Hence, we sometimes draw the second terminal as ground for convenience.

The equivalent circuits for logic gates are assembled from the individual transistors. Figure 4.6 shows the equivalent circuit for a fanout-of-1 inverter with negligible wire capacitance. The unit inverters of Figure 4.6(a) are composed from an nMOS transistor of unit size and a pMOS transistor of twice unit

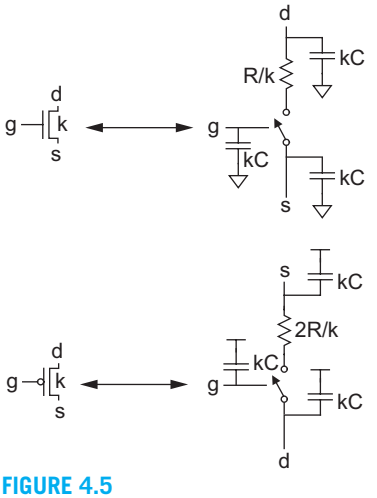


FIGURE 4.5
Equivalent circuits for transistors

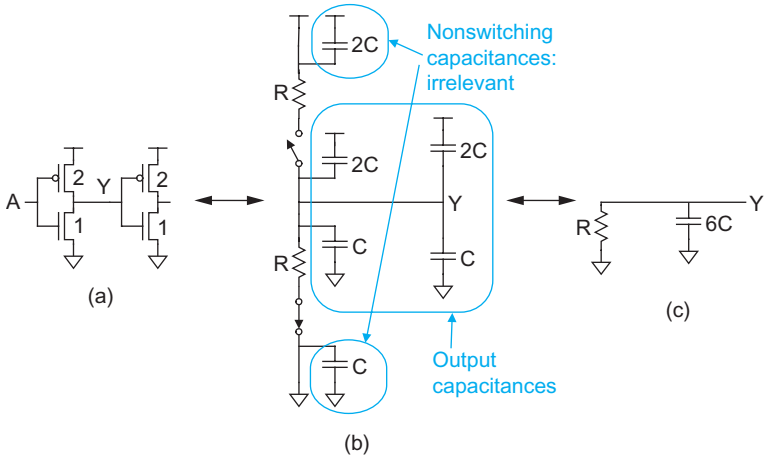


FIGURE 4.6 Equivalent circuit for an inverter

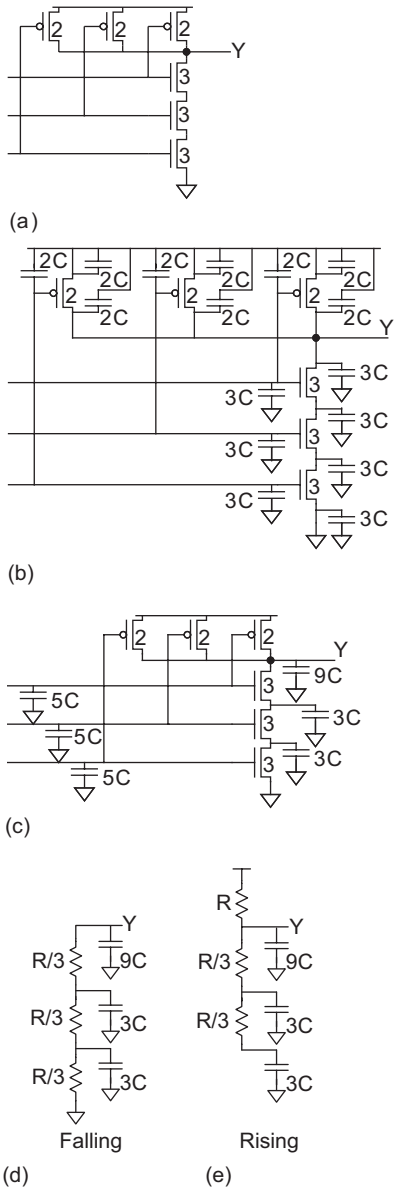


FIGURE 4.7 Equivalent circuits for a 3-input NAND gate

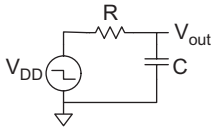


FIGURE 4.8
First-order RC system

width to achieve equal rise and fall resistance. Figure 4.6(b) gives an equivalent circuit, showing the first inverter driving the second inverter's gate. If the input A rises, the nMOS transistor will be ON and the pMOS OFF. Figure 4.6(c) illustrates this case with the switches removed. The capacitors shorted between two constant supplies are also removed because they are not charged or discharged. The total capacitance on the output Y is $6C$.

Example 4.2

Sketch a 3-input NAND gate with transistor widths chosen to achieve effective rise and fall resistance equal to that of a unit inverter (R). Annotate the gate with its gate and diffusion capacitances. Assume all diffusion nodes are contacted. Then sketch equivalent circuits for the falling output transition and for the worst-case rising output transition.

SOLUTION: Figure 4.7(a) shows such a gate. The three nMOS transistors are in series so the resistance is three times that of a single transistor. Therefore, each must be three times unit width to compensate. In other words, each transistor has resistance $R/3$ and the series combination has resistance R. The two pMOS transistors are in parallel. In the worst case (with one of the inputs low), only one of the pMOS transistors is ON. Therefore, each must be twice unit width to have resistance R.

Figure 4.7(b) shows the capacitances. Each input presents five units of gate capacitance to whatever circuit drives that input. Notice that the capacitors on source diffusions attached to the rails have both terminals shorted together so they are irrelevant to circuit operation. Figure 4.7(c) redraws the gate with these capacitances deleted and the remaining capacitances lumped to ground.

Figure 4.7(d) shows the equivalent circuit for the falling output transition. The output pulls down through the three series nMOS transistors. Figure 4.7(e) shows the equivalent circuit for the rising output transition. In the worst case, the upper two inputs are 1 and the bottom one falls to 0. The output pulls up through a single pMOS transistor. The upper two nMOS transistors are still on, so the diffusion capacitance between the series nMOS transistors must also be discharged.

4.3.4 Transient Response

Now, consider applying the RC model to estimate the step response of the first-order system shown in Figure 4.8. This system is a good model of an inverter sized for equal rise and fall delays. The system has a transfer function

$$H(s) = \frac{1}{1 + sRC} \tag{4.6}$$

and a step response

$$V_{out}(t) = V_{DD} e^{-t/\tau} \tag{4.7}$$

where $\tau = RC$. The propagation delay is the time at which V_{out} reaches $V_{DD}/2$, as shown in Figure 4.9.

$$t_{pd} = RC \ln 2 \tag{4.8}$$

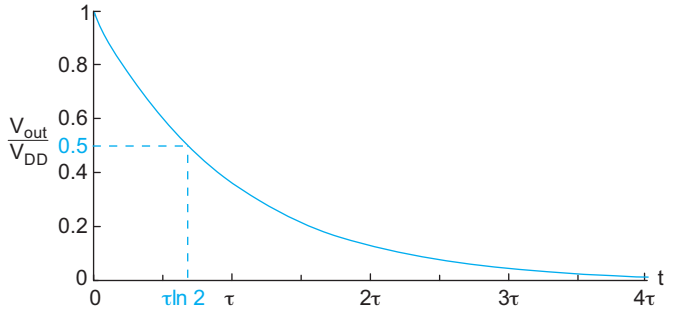


FIGURE 4.9 First-order step response

The factor of ln 2 = 0.69 is cumbersome. The effective resistance R is an empirical parameter anyway, so it is preferable to incorporate the factor of ln 2 to define a new effective resistance $R' = R \ln 2$. Now the propagation delay is simply $R'C$. For the sake of convenience, we usually drop the prime symbols and just write

$$t_{pd} = RC \tag{4.9}$$

where the effective resistance R is chosen to give the correct delay.

Figure 4.10 shows a second-order system. R_1 and R_2 might model the two series nMOS transistors in a NAND gate or an inverter driving a long wire with non-negligible resistance. The transfer function is

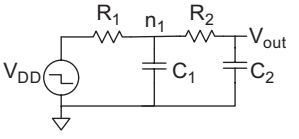


FIGURE 4.10 Second-order RC system

$$H(s) = \frac{1}{1 + s\left[R_1C_1 + (R_1 + R_2)C_2\right] + s^2R_1C_1R_2C_2} \tag{4.10}$$

The function has two real poles and the step response is

$$V_{out}(t) = V_{DD}\frac{\tau_1 e^{-t/\tau_1} - \tau_2 e^{-t/\tau_2}}{\tau_1 - \tau_2} \tag{4.11}$$

with

$$\tau_{1,2} = \frac{R_1C_1 + (R_1 + R_2)C_2}{2}\left(1 \pm \sqrt{1 - \frac{4R^*C^*}{\left[1 + (1 + R^*)C^*\right]^2}}\right) \tag{4.12}$$

$$R^* = \frac{R_2}{R_1};\ C^* = \frac{C_2}{C_1}$$

EQ (4.12) is so complicated that it defeats the purpose of simplifying a CMOS circuit into an equivalent RC network. However, it can be further approximated as a first-order system with a single time constant:

$$\tau = \tau_1 + \tau_2 = R_1C_1 + \left(R_1 + R_2\right)C_2 \tag{4.13}$$

This approximation works best when one time constant is significantly bigger than the other [Horowitz84]. For example, if $R_1 = R_2 = R$ and $C_1 = C_2 = C$, then $\tau_1 = 2.6\ RC$, $\tau_2 = 0.4\ RC$, $\tau = 3\ RC$ and the second-order response and its first-order approximation are shown in Figure 4.11. The error in estimated propagation delay from the first-order approximation is less than 7%. Even in the worst case, where the two time constants are equal, the error is less than 15%. The single time constant is a bad description of the behavior of intermediate nodes. For example, the response at n_1 cannot be described well by a single time constant. However, CMOS designers are primarily interested in the delay to the output of a gate, where the approximation works well. In the next section, we will see how to find a simple single time constant approximation for general RC tree circuits using the Elmore delay model.

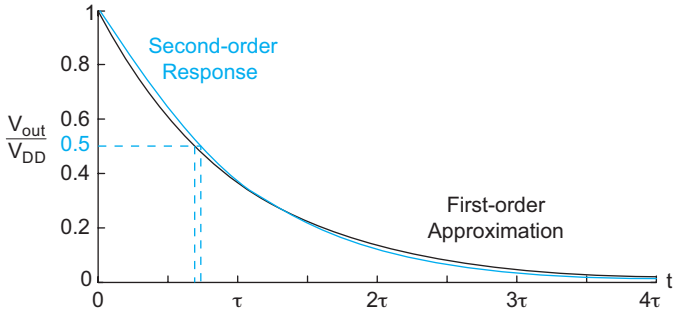


FIGURE 4.11 Comparison of second-order response to first-order approximation

4.3.5 Elmore Delay

In general, most circuits of interest can be represented as an *RC tree*, i.e., an RC circuit with no loops. The root of the tree is the voltage source and the leaves are the capacitors at the ends of the branches. The Elmore delay model [Elmore48] estimates the delay from a source switching to one of the leaf nodes changing as the sum over each node i of the capacitance C_i on the node, multiplied by the effective resistance R_{is} on the shared path from the source to the node and the leaf. Application of Elmore delay is best illustrated through examples.

$$t_{pd} = \sum_i R_{is}C_i \tag{4.14}$$

Example 4.3

Compute the Elmore delay for V_{out} in the 2nd order RC system from Figure 4.10.

SOLUTION: The circuit has a source and two nodes. At node n_1, the capacitance is C_1 and the resistance to the source is R_1. At node V_{out}, the capacitance is C_2 and the resistance to the source is $(R_1 + R_2)$. Hence, the Elmore delay is $t_{pd} = R_1C_1 + (R_1 + R_2)C_2$, just as the single time constant predicted in EQ (4.13). Note that the effective resistances should account for the factor of ln 2.

Example 4.4

Estimate t_{pd} for a unit inverter driving m identical unit inverters.

SOLUTION: Figure 4.12 shows an equivalent circuit for the falling transition. Each load inverter presents $3C$ units of gate capacitance, for a total of $3mC$. The output node also sees a capacitance of $3C$ from the drain diffusions of the driving inverter. This capacitance is called *parasitic* because it is an undesired side-effect of the need to make the drain large enough to contact. The parasitic capacitance is independent of the load that the inverter is driving. Hence, the total capacitance is $(3 + 3m)C$. The resistance is R, so the Elmore delay is $t_{pd} = (3 + 3m)RC$. The equivalent circuit for the rising transition gives the same results.

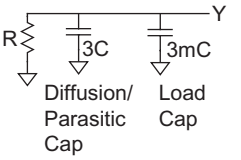


FIGURE 4.12 Equivalent circuit for inverter

Example 4.5

Repeat Example 4.4 if the driver is w times unit size.

SOLUTION: Figure 4.13 shows the equivalent circuit. The driver transistors are w times as wide, so the effective resistance decreases by a factor of w . The diffusion capacitance increases by a factor of w. The Elmore delay is $t_{pd} = ((3w + 3m)C)(R/w) = (3 + 3m/w)RC$.

Define the *fanout* of the gate, h, to be the ratio of the load capacitance to the input capacitance. (Diffusion capacitance is not counted in the fanout.) The load capacitance is $3mC$. The input capacitance is $3wC$. Thus, the inverter has a fanout of $h = m/w$ and the delay can be written as $(3 + 3h)RC$.

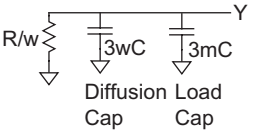


FIGURE 4.13 Equivalent circuit for wider inverter

Example 4.6

If a unit transistor has $R = 10\ \text{k}\Omega$ and $C = 0.1$ fF in a 65 nm process, compute the delay, in picoseconds, of the inverter in Figure 4.14 with a fanout of $h = 4$.

SOLUTION: The RC product in the 65 nm process is $(10\ \text{k}\Omega)(0.1\ \text{fF}) = 1$ ps. For $h = 4$, the delay is $(3 + 3h)(1\ \text{ps}) = 15$ ps. This is called the *fanout-of-4 (FO4) inverter delay* and is representative of gate delays in a typical circuit. Remember that a picosecond is a trillionth of a second. The inverter can switch about 66 billion times per second. This stunning speed partially explains the fantastic capabilities of integrated circuits.

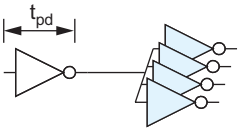


FIGURE 4.14 Fanout-of-4 (FO4) inverter

It is often helpful to express delay in a process-independent form so that circuits can be compared based on topology rather than speed of the manufacturing process. Moreover, with a process-independent measure for delay, knowledge of circuit speeds gained while working in one process can be carried over to a new process. Observe that the delay of an ideal fanout-of-1 inverter with no parasitic capacitance is $\tau = 3RC$ [1] [Sutherland99]. We denote the normalized delay d relative to this inverter delay:

$$d = \frac{t_{pd}}{\tau} \tag{4.15}$$

[1] Do not confuse this definition of $\tau = 3RC$, the delay of a parasitic-free fanout-of-1 inverter, with Mead and Conway's definition [Mead80] $\tau = RC$, the delay of an nMOS transistor driving its own gate, or with the use of τ as an arbitrary time constant. For the remainder of this text, $\tau = 3RC$.

Hence, the delay of a fanout-of-h inverter can be written in normalized form as $d = h + 1$, assuming that diffusion capacitance approximately equals gate capacitance. An FO4 inverter has a delay of 5τ. If diffusion capacitance were slightly higher or lower, the FO4 delay would change by only a small amount. Thus, circuit delay measured in FO4 delays is nearly constant from one process to another.[2]

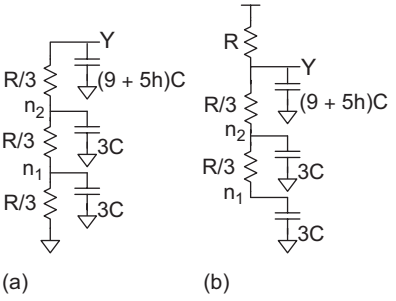


FIGURE 4.15 Equivalent circuits for loaded gate

Example 4.7

Estimate t_{pdf} and t_{pdr} for the 3-input NAND gate from Example 4.2 if the output is loaded with h identical NAND gates.

SOLUTION: Each NAND gate load presents 5 units of capacitance on a given input. Figure 4.15(a) shows the equivalent circuit including the load for the falling transition. Node n_1 has capacitance $3C$ and resistance of $R/3$ to ground. Node n_2 has capacitance $3C$ and resistance $(R/3 + R/3)$ to ground. Node Y has capacitance $(9 + 5h)C$ and resistance $(R/3 + R/3 + R/3)$ to ground. The Elmore delay for the falling output is the sum of these RC products, $t_{pdf} = (3C)(R/3) + (3C)(R/3 + R/3) + ((9 + 5h)C)(R/3 + R/3 + R/3) = (12 + 5h)RC$.

Figure 4.15(b) shows the equivalent circuit for the falling transition. In the worst case, the two inner inputs are 1 and the outer input falls. Y is pulled up to V_{DD} through a single pMOS transistor. The ON nMOS transistors contribute parasitic capacitance that slows the transiton. Node Y has capacitance $(9 + 5h)C$ and resistance R to the V_{DD} supply. Node n_2 has capacitance $3C$. The relevant resistance is only R, not $(R + R/3)$, because the output is being charged only through R. This is what is meant by the resistance on the shared path from the source (V_{DD}) to the node (n_2) and the leaf (Y). Similarly, node n_1 has capacitance $3C$ and resistance R. Hence, the Elmore delay for the rising output is $t_{pdr} = (15 + 5h)RC$. The $R/3$ resistances do not contribute to this delay. Indeed, they shield the diffusion capacitances, which don't have to charge all the way up before Y rises. Hence, the Elmore delay is conservative and the actual delay is somewhat faster.

Although the gate has equal resistance pulling up and down, the delays are not quite equal because of the capacitances on the internal nodes.

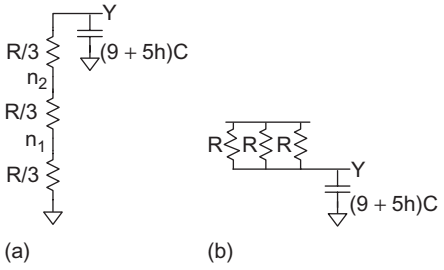


FIGURE 4.16 Equivalent circuits for contamination delay

Example 4.8

Estimate the contamination delays t_{cdf} and t_{cdr} for the 3-input NAND gate from Example 4.2 if the output is loaded with h identical NAND gates.

SOLUTION: The contamination delay is the fastest that the gate might switch. For the falling transition, the best case is that the bottom two nMOS transistors are already ON when the top one turns ON. In such a case, the diffusion capacitances on n_1 and n_2 have already been discharged and do not contribute to the delay. Figure 4.16(a) shows the equivalent circuit and the delay is $t_{cdf} = (9 + 5h)RC$.

[2]This assumes that the circuit is dominated by gate delay. The RC delay of long wires does not track well with the gate delay, as will be explored in Chapter 6.

For the rising transition, the best case is that all three pMOS transistors turn on simultaneously. The nMOS transistors turn OFF, so n_1 and n_2 are not connected to the output and do not contribute to delay. The parallel transistors deliver three times as much current, as shown in Figure 4.16(b), so the delay is $t_{cdr} = (3 + (5/3)h)RC$.

In all of the Examples, the delay consists of two components. The *parasitic delay* is the time for a gate to drive its own internal diffusion capacitance. Boosting the width of the transistors decreases the resistance but increases the capacitance so the parasitic delay is ideally independent of the gate size.[3] The *effort delay* depends on the ratio h of external load capacitance to input capacitance and thus changes with transistor widths. It also depends on the complexity of the gate. The capacitance ratio is called the *fanout* or *electrical effort* and the term indicating gate complexity is called the *logical effort*. For example, an inverter has a delay of $d = h + 1$, so the parasitic delay is 1 and the logical effort is also 1. The NAND3 has a worst case delay of $d = (5/3)h + 5$. Thus, it has a parasitic delay of 5 and a logical effort of 5/3. These delay components will be explored further in Section 4.4.

4.3.6 Layout Dependence of Capacitance

In a good layout, diffusion nodes are shared wherever possible to reduce the diffusion capacitance. Moreover, the uncontacted diffusion nodes between series transistors are usually smaller than those that must be contacted. Such uncontacted nodes have less capacitance (see Sections 2.3.3 and 8.4.4), although we will neglect the difference for hand calculations. A conservative method of estimating capacitances before layout is to assume uncontacted diffusion between series transistors and contacted diffusion on all other nodes. However, a more accurate estimate can be made once the layout is known.

Example 4.9

Figure 4.17(a) shows a layout of the 3-input NAND gate. A single drain diffusion region is shared between two of the pMOS transistors. Estimate the actual diffusion capacitance from the layout.

SOLUTION: Figure 4.17(b) redraws the schematic with these capacitances lumped to ground. The output node has the following diffusion capacitances: $3C$ from the nMOS transistor drain, $2C$ from the isolated pMOS transistor drain, and $2C$ from a pair of pMOS drains that share a contact. Thus, the actual diffusion capacitance on the output is $7C$, rather than $9C$ predicted in Figure 4.15.

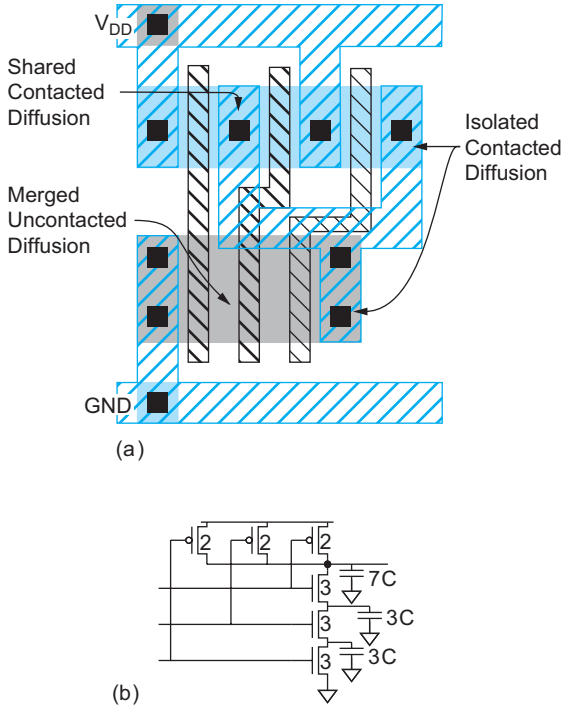


FIGURE 4.17 3-input NAND annotated with diffusion capacitances extracted from the layout

[3]Gates with wider transistors may use layout tricks so the diffusion capacitance increases less than linearly with width, slightly decreasing the parasitic delay of large gates as discussed in Section 4.3.6.

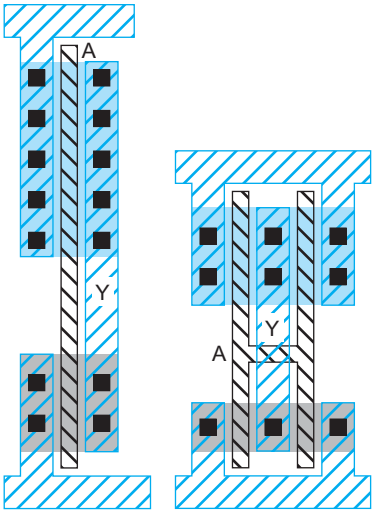


FIGURE 4.18 Layout styles: (a) conventional, (b) folded

The diffusion capacitance can also be decreased by *folding* wide transistors. Figure 4.18(a) shows a conventional layout of a 24/12 λ inverter. Because a unit (4 λ) transistor has diffusion capacitance C, the inverter has a total diffusion capacitance of $9C$. The folded layout in Figure 4.18(b) constructs each transistor from two parallel devices of half the width. Observe that the diffusion area has shrunk by a factor of two, reducing the diffusion capacitance to $4.5C$. In general, folded layouts offer lower parasitic delay than unfolded layouts. The folded layout may also fit better in a standard cell of limited height, and the shorter polysilicon lines have lower resistance. For these reasons, wide transistors are folded whenever possible.

In some nanometer processes (generally 45 nm and below), transistor gates are restricted to a limited choice of pitches to improve manufacturability and reduce variability. For example, the spacing between polysilicon for gates may always be the contacted transistor pitch, even if no contact is required. Moreover, using a single standard transistor width may reduce variability.

4.3.7 Determining Effective Resistance



The effective resistance can be determined through simulation or analysis. Section 8.4.5 explains the simulation technique, which is most accurate. This section, however, offers an analysis that provides more insight into the relationship of resistance to other parameters.

Recall that the effective resistance is the average value of V_{ds} / I_{ds} of a transistor during a switching event. As mentioned in Section 4.3.4, the resistance is scaled by a factor of ln 2 so that propagation delay can be written as an RC product. For the step response of a rising input, we are interested in the time for the output to discharge from V_{DD} to $V_{DD} / 2$ through an nMOS transistor. If the transistor is sufficiently velocity-saturated that $V_{\text{dsat}} < V_{DD} / 2$, then the transistor will remain in the saturation region throughout this transition and the current is roughly constant at I_{dsat}. In such a case, the effective resistance is

$$R_{\text{step}} = \frac{\ln 2}{V_{DD}/2} \int_{V_{DD}/2}^{V_{DD}} \frac{V}{I_{\text{dsat}}} dV = \frac{3 \ln 2}{4} \frac{V_{DD}}{I_{\text{dsat}}} \approx \frac{V_{DD}}{2 I_{\text{dsat}}} \qquad (4.16)$$

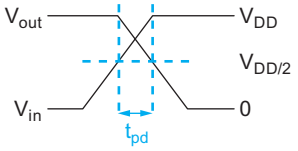


FIGURE 4.19 Propagation delay with input and output approximated as ramps

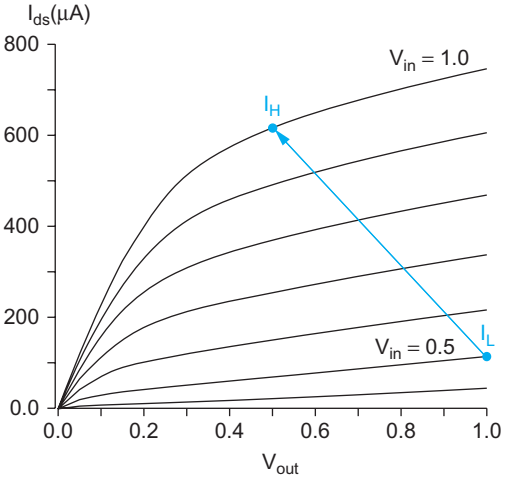


FIGURE 4.20 Approximate switching trajectory

Channel length modulation and DIBL cause the current to decrease somewhat with V_{ds} in a real transistor, slightly increasing the effective resistance.

More importantly, the input has a nonzero rise time and we are interested in the time from when the input rises through $V_{DD} / 2$ until the output falls through $V_{DD} / 2$. Assume that the input and output slopes are equal and that the output starts to fall when the input passes through $V_{DD} / 2$. Then, the output will reach $V_{DD} / 2$ when the input reaches V_{DD}, as shown in Figure 4.19.

Define the transistor current to be I_L at the start of the transition (when $V_{gs} = V_{DD} / 2$, $V_{ds} = V_{DD}$) and I_H at the end of the transition (when $V_{gs} = V_{DD}$, $V_{ds} = V_{DD} / 2$), as shown in Figure 4.20. Then, the transistor can be approximated during the switching event as a current source I_{eff} that is the average of these two extremes [Na02]:

$$I_{\text{eff}} = \frac{I_H + I_L}{2} \qquad (4.17)$$

The time for the output to discharge to $V_{DD}/2$ is thus:

$$t_{pd} = \frac{CV_{DD}}{2I_{\text{eff}}} \tag{4.18}$$

Equating this to $t_{pd} = RC$ gives

$$R = \frac{V_{DD}}{2I_{\text{eff}}} = \frac{V_{DD}}{I_H + I_L} \tag{4.19}$$

4.4 Linear Delay Model

The RC delay model showed that delay is a linear function of the fanout of a gate. Based on this observation, designers further simplify delay analysis by characterizing a gate by the slope and *y*-intercept of this function. In general, the normalized delay of a gate can be expressed in units of τ as

$$d = f + p \tag{4.20}$$

p is the *parasitic delay* inherent to the gate when no load is attached. f is the *effort delay* or *stage effort* that depends on the complexity and fanout of the gate:

$$f = gh \tag{4.21}$$

The complexity is represented by the *logical effort*, g [Sutherland99]. An inverter is defined to have a logical effort of 1. More complex gates have greater logical efforts, indicating that they take longer to drive a given fanout. For example, the logical effort of the 3-input NAND gate from the previous example is 5/3. A gate driving h identical copies of itself is said to have a *fanout* or *electrical effort* of h. If the load does not contain identical copies of the gate, the electrical effort can be computed as

$$h = \frac{C_{\text{out}}}{C_{\text{in}}} \tag{4.22}$$

where C_{out} is the capacitance of the external load being driven and C_{in} is the input capacitance of the gate.[4]

Figure 4.21 plots normalized delay vs. electrical effort for an idealized inverter and 3-input NAND gate. The *y*-intercepts indicate the parasitic delay, i.e., the delay when the gate drives no load. The slope of the lines is the logical effort. The inverter has a slope of 1 by definition. The NAND has a slope of 5/3.

The remainder of this section explores how to estimate the logical effort and parasitic delay and how to use the linear delay model.

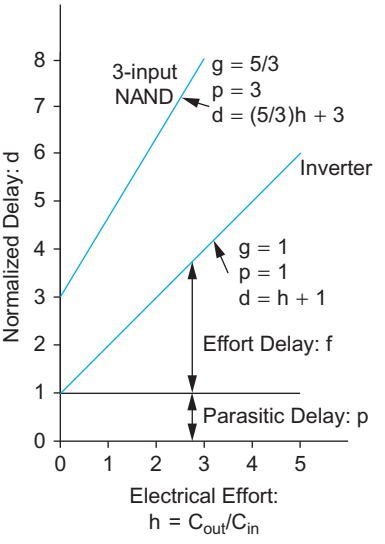


FIGURE 4.21
Normalized delay vs. fanout

[4]Some board-level designers say a device has a fanout of h when it drives h other devices, even if the other devices have different capacitances. This definition would not be useful for calculating delay and is best avoided in VLSI design. The term *electrical effort* avoids this potential confusion and emphasizes the parallels with logical effort.

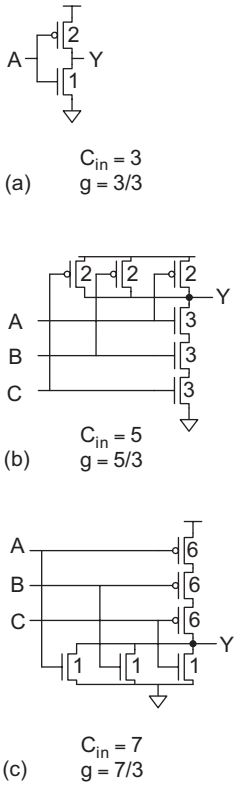


FIGURE 4.22 Logic gates sized for unit resistance

4.4.1 Logical Effort

Logical effort of a gate is defined as *the ratio of the input capacitance of the gate to the input capacitance of an inverter that can deliver the same output current*. Equivalently, logical effort indicates how much worse a gate is at producing output current as compared to an inverter, given that each input of the gate may only present as much input capacitance as the inverter.

Logical effort can be measured in simulation from delay vs. fanout plots as the ratio of the slope of the delay of the gate to the slope of the delay of an inverter, as will be discussed in Section 8.5.3. Alternatively, it can be estimated by sketching gates. Figure 4.22 shows inverter, 3-input NAND, and 3-input NOR gates with transistor widths chosen to achieve unit resistance, assuming pMOS transistors have twice the resistance of nMOS transistors.[5] The inverter presents three units of input capacitance. The NAND presents five units of capacitance on each input, so the logical effort is 5/3. Similarly, the NOR presents seven units of capacitance, so the logical effort is 7/3. This matches our expectation that NANDs are better than NORs because NORs have slow pMOS transistors in series.

Table 4.2 lists the logical effort of common gates. The effort tends to increase with the number of inputs. NAND gates are better than NOR gates because the series transistors are nMOS rather than pMOS. Exclusive-OR gates are particularly costly and have different logical efforts for different inputs. An interesting case is that multiplexers built from ganged tristates, as shown in Figure 1.29(b), have a logical effort of 2 independent of the number of inputs. This might at first seem to imply that very large multiplexers are just as fast as small ones. However, the parasitic delay does increase with multiplexer size; hence, it is generally fastest to construct large multiplexers out of trees of 4-input multiplexers [Sutherland99].

TABLE 4.2 Logical effort of common gates

Gate Type	Number of Inputs				
	1	2	3	4	*n*
inverter	1				
NAND		4/3	5/3	6/3	$(n+2)/3$
NOR		5/3	7/3	9/3	$(2n+1)/3$
tristate, multiplexer	2	2	2	2	2
XOR, XNOR		4, 4	6, 12, 6	8, 16, 16, 8	

4.4.2 Parasitic Delay

The parasitic delay of a gate is the delay of the gate when it drives zero load. It can be estimated with RC delay models. A crude method good for hand calculations is to count only diffusion capacitance on the output node. For example, consider the gates in Figure 4.22, assuming each transistor on the output node has its own drain diffusion contact. Transistor widths were chosen to give a resistance of R in each gate. The inverter has three units of diffusion capacitance on the output, so the parasitic delay is $3RC = \tau$. In other words,

[5]This assumption is made throughout the book. Exercises 4.19–4.20 explore the effects of different relative resistances (see also [Sutherland99]). The overall conclusions do not change very much, so the simple model is good enough for most hand estimates. A simulator or static timing analyzer should be used when more accurate results are required.

the normalized parasitic delay is 1. In general, we will call the normalized parasitic delay p_{inv}. p_{inv} is the ratio of diffusion capacitance to gate capacitance in a particular process. It is usually close to 1 and will be considered to be 1 in many examples for simplicity. The 3-input NAND and NOR each have 9 units of diffusion capacitance on the output, so the parasitic delay is three times as great ($3p_{inv}$, or simply 3). Table 4.3 estimates the parasitic delay of common gates. Increasing transistor sizes reduces resistance but increases capacitance correspondingly, so parasitic delay is, on first order, independent of gate size. However, wider transistors can be folded and often see less than linear increases in internal wiring parasitic capacitance, so in practice, larger gates tend to have slightly lower parasitic delay.

TABLE 4.3 Parasitic delay of common gates

Gate Type	Number of Inputs				
	1	2	3	4	*n*
inverter	1				
NAND		2	3	4	*n*
NOR		2	3	4	*n*
tristate, multiplexer	2	4	6	8	2*n*

This method of estimating parasitic delay is obviously crude. More refined estimates use the Elmore delay counting internal parasitics, as in Example 4.7, or extract the delays from simulation. The parasitic delay also depends on the ratio of diffusion capacitance to gate capacitance. For example, in a silicon-on-insulator process in which diffusion capacitance is much less, the parasitic delays will be lower. While knowing the parasitic delay is important for accurately estimating gate delay, we will see in Section 4.5 that the best transistor sizes for a particular circuit are only weakly dependent on parasitic delay. Hence, crude estimates tend to be sufficient to reach a good circuit design.

Nevertheless, it is important to realize that parasitic delay grows more than linearly with the number of inputs in a real NAND or NOR circuit. For example, Figure 4.23 shows a model of an *n*-input NAND gate in which the upper inputs were all 1 and the bottom input rises. The gate must discharge the diffusion capacitances of all of the internal nodes as well as the output. The Elmore delay is

$$t_{pd} = R(3nC) + \sum_{i=1}^{n-1}\left(\frac{iR}{n}\right)(nC) = \left(\frac{n^2}{2} + \frac{5}{2}n\right)RC \qquad \textbf{(4.23)}$$

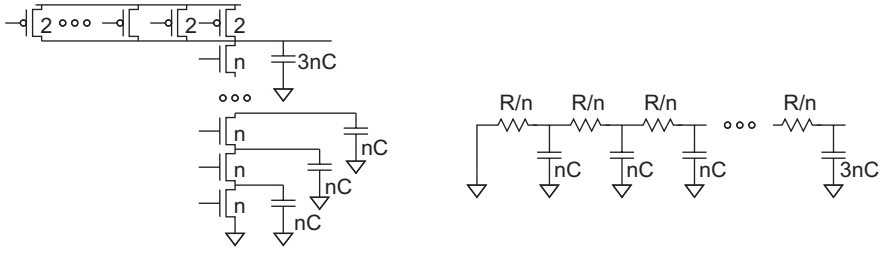


FIGURE 4.23 *n*-input NAND gate parasitic delay

This delay grows quadratically with the number of series transistors n, indicating that beyond a certain point it is faster to split a large gate into a cascade of two smaller gates. We will see in Section 4.4.6.5 that the coefficient of the n^2 term tends to be even larger in real circuits than in this simple model because of gate-source capacitance. In practice, it is rarely advisable to construct a gate with more than four or possibly five series transistors. When building large fan-in gates, trees of NAND gates are better than NOR gates because the NANDs have lower logical effort.

4.4.3 Delay in a Logic Gate

Consider two examples of applying the linear delay model to logic gates.

Example 4.10

Use the linear delay model to estimate the delay of the fanout-of-4 (FO4) inverter from Example 4.6. Assume the inverter is constructed in a 65 nm process with $\tau = 3$ ps.

SOLUTION: The logical effort of the inverter is $g = 1$, by definition. The electrical effort is 4 because the load is four gates of equal size. The parasitic delay of an inverter is $p_{\text{inv}} \approx 1$. The total delay is $d = gh + p = 1 \times 4 + 1 = 5$ in normalized terms, or $t_{pd} = 15$ ps in absolute terms.

Often path delays are expressed in terms of FO4 inverter delays. While not all designers are familiar with the τ notation, most experienced designers do know the delay of a fanout-of-4 inverter in the process in which they are working. τ can be estimated as 0.2 FO4 inverter delays. Even if the ratio of diffusion capacitance to gate capacitance changes so $p_{\text{inv}} = 0.8$ or 1.2 rather than 1, the FO4 inverter delay only varies from 4.8 to 5.2. Hence, the delay of a gate-dominated logic block expressed in terms of FO4 inverters remains relatively constant from one process to another even if the diffusion capacitance does not.

As a rough rule of thumb, the FO4 delay for a process (in picoseconds) is 1/3 to 1/2 of the drawn channel length (in nanometers). For example, a 65 nm process with a 50 nm channel length may have an FO4 delay of 16–25 ps. Delay is highly sensitive to process, voltage, and temperature variations, as will be examined in Section 7.2. The FO4 delay is usually quoted assuming typical process parameters and worst-case environment (low power supply voltage and high temperature).

Example 4.11

A ring oscillator is constructed from an odd number of inverters, as shown in Figure 4.24. Estimate the frequency of an N-stage ring oscillator.

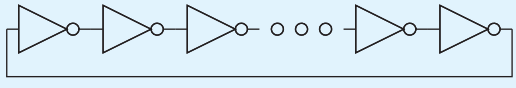


FIGURE 4.24 Ring oscillator

SOLUTION: The logical effort of the inverter is $g = 1$, by definition. The electrical effort of each inverter is also 1 because it drives a single identical load. The parasitic delay is also 1. The delay of each stage is $d = gh + p = 1 \times 1 + 1 = 2$. An N-stage ring oscillator

has a period of $2N$ stage delays because a value must propagate twice around the ring to regain the original polarity. Therefore, the period is $T = 2 \times 2N$. The frequency is the reciprocal of the period, $1/4N$.

A 31-stage ring oscillator in a 65 nm process has a frequency of $1/(4 \times 31 \times 3 \text{ ps}) = 2.7$ GHz.

Note that ring oscillators are often used as process monitors to judge if a particular chip is faster or slower than nominally expected. One of the inverters should be replaced with a NAND gate to turn the ring off when not in use. The output can be routed to an external pad, possibly through a test multiplexer. The oscillation frequency should be low enough (e.g., 100 MHz) that the path to the outside world does not attenuate the signal too badly.

4.4.4 Drive

A good standard cell library contains multiple sizes of each common gate. The sizes are typically labeled with their drive. For example, a unit inverter may be called inv_1x. An inverter of eight times unit size is called inv_8x. A 2-input NAND that delivers the same current as the inverter is called nand2_1x.

It is often more intuitive to characterize gates by their drive, x, rather than their input capacitance. If we redefine a unit inverter to have one unit of input capacitance, then the drive of an arbitrary gate is

$$x = \frac{C_{\text{in}}}{g} \tag{4.24}$$

Delay can be expressed in terms of drive as

$$d = \frac{C_{\text{out}}}{x} + p \tag{4.25}$$

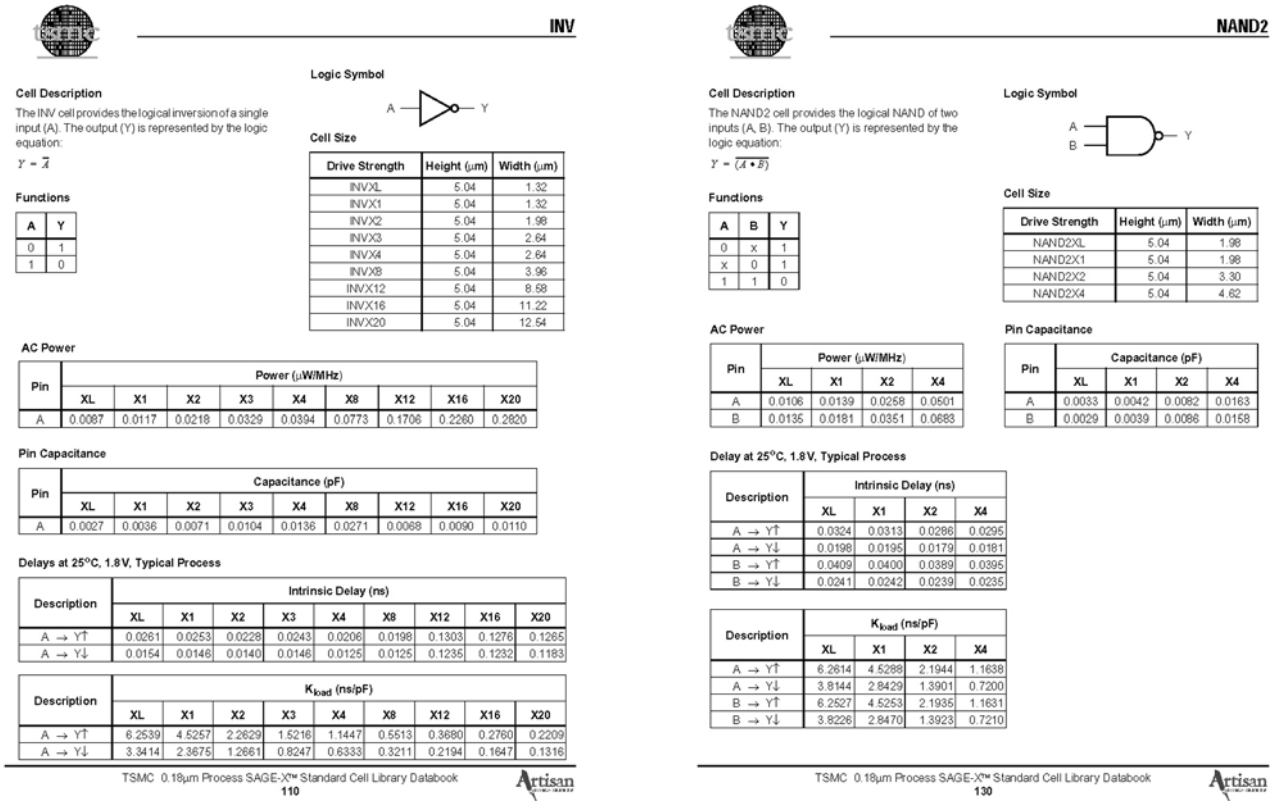
4.4.5 Extracting Logical Effort from Datasheets



When using a standard cell library, you can often extract logical effort of gates directly from the datasheets. For example, Figure 4.25 shows the INV and NAND2 datasheets from the Artisan Components library for the TSMC 180 nm process. The gates in the library come in various drive strengths. INVX1 is the unit inverter; INVX2 has twice the drive. INVXL has the same area as the unit inverter but uses smaller transistors to reduce power consumption on noncritical paths. The X12–X20 inverters are built from three stages of smaller inverters to give high drive strength and low input capacitance at the expense of greater parasitic delay.

From the datasheet, we see the unit inverter has an input capacitance of 3.6 fF. The rising and falling delays are specified separately. We will develop a notation for different delays in Section 9.2.1.5, but will use the average delay for now. The average *intrinsic* or parasitic delay is (25.3 + 14.6)/2 = 20.0 ps. The slope of the delay vs. load capacitance curve is the average of the rising and falling K_{load} values. An inverter driving a fanout of h will thus have a delay of

$$t_{pd} = 20.0 \text{ ps} + \left(3.6 \frac{\text{fF}}{\text{gate}}\right)(h \text{ gates})\left(\frac{4.53 + 2.37}{2} \frac{\text{ns}}{\text{pF}}\right) = (20.0 + 12.4h) \text{ ps} \tag{4.26}$$



INV

Cell Description

The INV cell provides the logical inversion of a single input (A). The output (Y) is represented by the logic equation:

$Y = \overline{A}$

Functions

A	Y
0	1
1	0

Logic Symbol

Cell Size

Drive Strength	Height (μm)	Width (μm)
INVXL	5.04	1.32
INVX1	5.04	1.32
INVX2	5.04	1.98
INVX3	5.04	2.64
INVX4	5.04	2.64
INVX8	5.04	3.96
INVX12	5.04	8.58
INVX16	5.04	11.22
INVX20	5.04	12.54

AC Power

Pin	Power (μW/MHz)								
	XL	X1	X2	X3	X4	X8	X12	X16	X20
A	0.0087	0.0117	0.0218	0.0329	0.0394	0.0773	0.1706	0.2260	0.2820

Pin Capacitance

Pin	Capacitance (pF)								
	XL	X1	X2	X3	X4	X8	X12	X16	X20
A	0.0027	0.0036	0.0071	0.0104	0.0136	0.0271	0.0068	0.0090	0.0110

Delays at 25°C, 1.8V, Typical Process

Description	Intrinsic Delay (ns)								
	XL	X1	X2	X3	X4	X8	X12	X16	X20
A → Y↑	0.0261	0.0253	0.0228	0.0243	0.0206	0.0198	0.1303	0.1276	0.1265
A → Y↓	0.0154	0.0146	0.0140	0.0146	0.0125	0.0125	0.1235	0.1232	0.1183

Description	K_{load} (ns/pF)								
	XL	X1	X2	X3	X4	X8	X12	X16	X20
A → Y↑	6.2539	4.5257	2.2629	1.5216	1.1447	0.5513	0.3680	0.2760	0.2209
A → Y↓	3.3414	2.3675	1.2661	0.8247	0.6333	0.3211	0.2194	0.1647	0.1316

TSMC 0.18μm Process SAGE-X™ Standard Cell Library Databook
110

NAND2

Cell Description

The NAND2 cell provides the logical NAND of two inputs (A, B). The output (Y) is represented by the logic equation:

$Y = \overline{(A \bullet B)}$

Functions

A	B	Y
0	x	1
x	0	1
1	1	0

Logic Symbol

Cell Size

Drive Strength	Height (μm)	Width (μm)
NAND2XL	5.04	1.98
NAND2X1	5.04	1.98
NAND2X2	5.04	3.30
NAND2X4	5.04	4.62

AC Power

Pin	Power (μW/MHz)			
	XL	X1	X2	X4
A	0.0106	0.0139	0.0258	0.0501
B	0.0135	0.0181	0.0351	0.0683

Pin Capacitance

Pin	Capacitance (pF)			
	XL	X1	X2	X4
A	0.0033	0.0042	0.0082	0.0163
B	0.0029	0.0039	0.0086	0.0158

Delay at 25°C, 1.8V, Typical Process

Description	Intrinsic Delay (ns)			
	XL	X1	X2	X4
A → Y↑	0.0324	0.0313	0.0286	0.0295
A → Y↓	0.0198	0.0195	0.0179	0.0181
B → Y↑	0.0409	0.0400	0.0389	0.0395
B → Y↓	0.0241	0.0242	0.0239	0.0235

Description	K_{load} (ns/pF)			
	XL	X1	X2	X4
A → Y↑	6.2614	4.5288	2.1944	1.1638
A → Y↓	3.8144	2.8429	1.3901	0.7200
B → Y↑	6.2527	4.5253	2.1935	1.1631
B → Y↓	3.8226	2.8470	1.3923	0.7210

TSMC 0.18μm Process SAGE-X™ Standard Cell Library Databook
130

Figure 4.25 Artisan Components cell library datasheets. Reprinted with permission.

The slope of the delay vs. fanout curve indicates $\tau = 12.4$ ps and the y-intercept indicates $p_{inv} = 20.0$ ps, or $(20.0/12.4) = 1.61$ in normalized terms. This is larger than the delay of 1 estimated earlier, probably because it includes capacitance of internal wires.

By a similar calculation, we find the X1 2-input NAND gate has an average delay from the inner (A) input of

$$t_{pd} = \left(\frac{31.3 + 19.5}{2}\right) \text{ps} + \left(4.2 \frac{\text{fF}}{\text{gate}}\right)(h \text{ gates}) \left(\frac{4.53 + 2.84}{2} \frac{\text{ns}}{\text{pF}}\right) = (25.4 + 15.5h) \text{ ps} \quad \textbf{(4.27)}$$

Thus, the parasitic delay is $(25.4/12.4) = 2.05$ and the logical effort is $(15.5/12.4) = 1.25$. The logical effort is slighly better than the theoretical 4/3 value, for reasons to be explored in Section 4.4.6.3. The parasitic delay from the outer (B) input is slightly higher, as expected. The parasitic delay and logical effort of the X2 and X4 gates are similar, confirming our model that logical effort should be independent of gate size for gates of reasonable sizes.

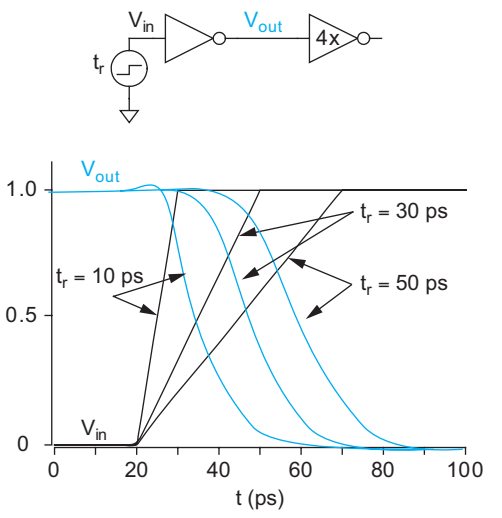


4.4.6 Limitations to the Linear Delay Model

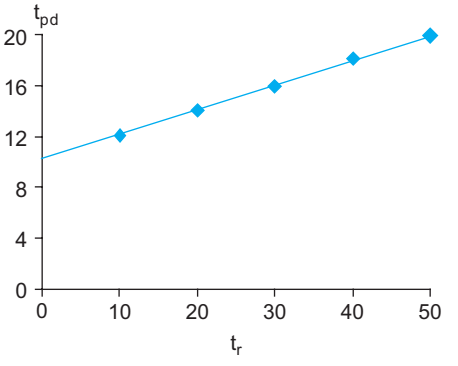
The linear delay model works remarkably well even in advanced technologies; for example, Figure 8.30 shows subpicosecond agreement in a 65 nm process assuming that input and output slopes are matched. Nevertheless, it also has limitations that should be understood when more accuracy is needed.

4.4.6.1 Input and Output Slope The largest source of error in the linear delay model is the input slope effect. Figure 4.26(a) shows a fanout-of-4 inverter driven by ramps with different slopes. Recall that the ON current increases with the gate voltage for an nMOS transistor. We say the transistor is *OFF* for $V_{gs} < V_t$, *fully ON* for $V_{gs} = V_{DD}$, and *partially ON* for intermediate gate voltages. As the rise time of the input increases, the delay also increases because the active transistor is not turned fully ON at once. Figure 4.26(b) plots average inverter propagation delay vs. input rise time. Notice that the delay vs. rise time data fits a straight line quite well [Hedenstierna87].

Accounting for slopes is important for accurate timing analysis (see Section 4.6), but is generally more complex than is worthwhile for hand calculations. Fortunately, we will see in Section 4.5 that circuits are fastest when each gate has the same effort delay and when that delay is roughly 4τ. Because slopes are related to edge rate, fast circuits tend to have relatively consistent slopes. If a cell library is characterized with these slopes, it will tend to be used in the regime in which it most accurately models delay.



(a)



(b)

FIGURE 4.26 SPICE simulation of slope effect on CMOS inverter delay

4.4.6.2 Input Arrival Times Another source of error in the linear delay model is the assumption that one input of a multiple-input gate switches while the others are completely stable. When two inputs to a series stack turn ON simultaneously, the delay will be slightly longer than predicted because both transistors are only partially ON during the initial part of the transition. When two inputs to a parallel stack turn ON simultaneously, the delay will be shorter than predicted because both transistors deliver current to the output. The delays are also slightly different depending on which input arrives first, as will be explored in Section 8.5.3.

4.4.6.3 Velocity Saturation The estimated logical efforts assume that N transistors in series must be N times as wide to give equal current. However, as discussed in Section 4.3.1, series transistors see less velocity saturation and hence have lower resistance than we estimated [Sakurai91].

To make a better estimate, observe that N transistors in series are equivalent to one transistor with N times the channel length. Substituting L and NL into EQ (2.28) shows that the ratio of I_{dsat} for two series transistors to that of a single transistor is

$$\frac{I_{dsat-N-series}}{I_{dsat}} = \frac{(V_{DD} - V_t) + V_c}{(V_{DD} - V_t) + NV_c} \tag{4.28}$$

In the limit that the transistors are not at all velocity saturated ($V_c >> V_{DD} - V_t$), the current ratio reduces to $1/N$ as predicted. In the limit that the transistors are completely velocity saturated, the current is independent of the number of series transistors.

Example 4.12

Determine the relative saturation current of 2- and 3-transistor nMOS and pMOS stacks in a 65 nm process. $V_{DD} = 1.0$ V and $V_t = 0.3$ V. Use $V_c = E_c L = 1.04$ V for nMOS devices and 2.22 V for pMOS devices.

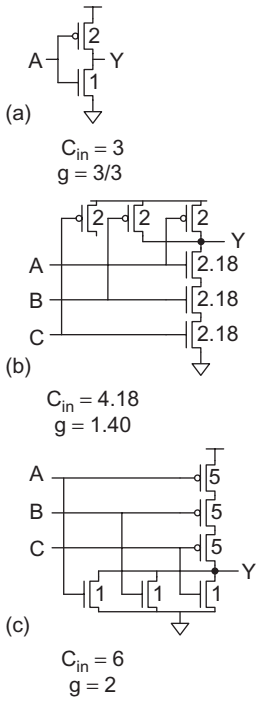


FIGURE 4.27 Logical effort estimates accounting for velocity saturation

SOLUTION: Applying EQ (4.28) gives a ratio of 0.63 for 2 nMOS transistors, 0.46 for 3 nMOS transistors, 0.57 for 2 pMOS transistors, and 0.40 for 3 pMOS transistors. The pMOS are closer to the ideal result of 0.5 and 0.33 because they experience less velocity saturation.

The transistors are scaled to deliver the same current as an inverter. Three series nMOS transistors must be 1/0.46 = 2.18 times as wide, rather than three times as wide. Three series pMOS transistors must be 2.5 times as wide. Figure 4.27 modifies Figure 4.22 to reflect velocity saturation. The logical efforts of the NAND and NOR are lower than originally predicted, and agree with the results obtained by curve-fitting SPICE simulations in Section 8.5.3.

4.4.6.4 Voltage Dependence Designers often need to predict how delay will vary if the supply or threshold voltage is changed. Recalling that delay is proportional to CV_{DD}/I and using the α-power law model of EQ (2.30) for I_{dsat}, we can estimate the scaling of the RC time constant and of gate delay as

$$\tau = k\frac{CV_{DD}}{\left(V_{DD}-V_t\right)^{\alpha}} \tag{4.29}$$

where k reflects process parameters.

Alternatively, using the straight line saturation current model from EQ (2.32) for velocity-saturated transistors, we obtain an even simpler estimate:

$$\tau = k\frac{CV_{DD}}{\left(V_{DD}-V_t^*\right)} = \frac{kC}{1-\frac{V_t^*}{V_{DD}}} \tag{4.30}$$

This model predicts that the supply voltage can be reduced without changing the delay of a velocity-saturated transistor so long as the threshold is reduced in proportion.

When $V_{DD} < V_t$, delay instead depends on the subthreshold current of EQ (2.45):

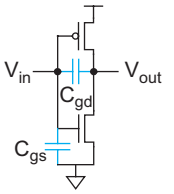
$$\tau = k\frac{CV_{DD}}{I_{off}10^{\frac{V_{DD}}{S}}} \tag{4.31}$$

4.4.6.5 Gate-Source Capacitance The examples in Section 4.3 assumed that gate capacitance terminates on a fixed supply rail. As discussed in Section 2.3.2, the bottom terminal of the gate oxide capacitor is the channel, which is primarily connected to the source when the transistor is ON. This means that as the source of a transistor changes value, charge is required to change the voltage on C_{gs}, adding to the delay for series stacks.

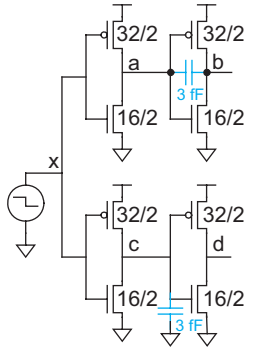
4.4.6.6 Bootstrapping Transistors also have some capacitance from gate to drain. This capacitance couples the input and output in an effect known as *bootstrapping*, which can be understood by examining Figure 4.28(a). Our models so far have only considered C_{in} (C_{gs}). This figure also considers C_{gd}, the gate to drain capacitance. In the case that the input is rising (the output starts high), the effective input capacitance is $C_{gs} + C_{gd}$. When the output starts to fall, the voltage across C_{gd} changes, requiring the input to supply additional current to charge C_{gd}. In other words, the impact of C_{gd} on gate capacitance is effectively doubled.

To illustrate the effect of the bootstrap capacitance on a circuit, Figure 4.28(b) shows two inverter pairs. The top pair has an extra bit of capacitance between the input and output of the second inverter. The bottom pair has the same amount of extra capacitance from input to ground. When *x* falls, nodes *a* and *c* begin to rise (Figure 4.28(c)). At first, both nodes see approximately the same capacitance, consisting of the two transistors and the extra 3 fF. As node *a* rises, it initially bumps up *b* or "lifts *b* by its own bootstraps." Eventually the nMOS transistors turn ON, pulling down *b* and *d*. As *b* falls, it tugs on *a* through the capacitor, leading to the slow final transition visible on node *a*. Also observe that *b* falls later than *d* because of the extra charge that must be supplied to discharge the bootstrap capacitor. In summary, the extra capacitance has a greater effect when connected between input and output as compared to when it is connected between input and ground.

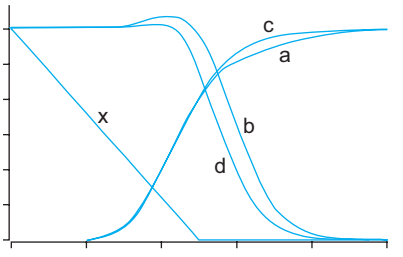
Because C_{gd} is fairly small, bootstrapping is only a mild annoyance in digital circuits. However, if the inverter is biased in its linear region near $V_{DD}/2$, the C_{gd} is multiplied by the large gain of the inverter. This is known as the *Miller effect* and is of major importance in analog circuits.



(a)



(b)



(c)

FIGURE 4.28 The effect of bootstrapping on inverter delay and waveform shape

4.5 Logical Effort of Paths

Designers often need to choose the fastest circuit topology and gate sizes for a particular logic function and to estimate the delay of the design. As has been stated, simulation or timing analysis are poor tools for this task because they only determine how fast a particular implementation will operate, not whether the implementation can be modified for better results and if so, what to change. Inexperienced designers often end up in the "simulate and tweak" loop involving minor changes and many fruitless simulations. The method of Logical Effort [Sutherland99] provides a simple method "on the back of an envelope" to choose the best topology and number of stages of logic for a function. Based on the linear delay model, it allows the designer to quickly estimate the best number of stages for a path, the minimum possible delay for the given topology, and the gate sizes that achieve this delay. The techniques of Logical Effort will be revisited throughout this text to understand the delay of many types of circuits.

4.5.1 Delay in Multistage Logic Networks

Figure 4.29 shows the logical and electrical efforts of each stage in a multistage path as a function of the sizes of each stage. The path of interest (the only path in this case) is marked with the dashed blue line. Observe that logical effort is independent of size, while electrical effort depends on sizes. This section develops some metrics for the path as a whole that are independent of sizing decisions.

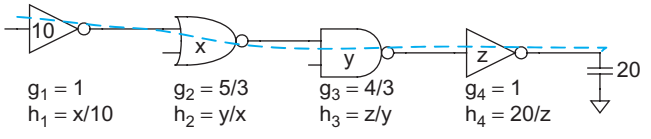


FIGURE 4.29 Multistage logic network

The *path logical effort* G can be expressed as the products of the logical efforts of each stage along the path.

$$G = \prod g_i \tag{4.32}$$

The *path electrical effort* H can be given as the ratio of the output capacitance the path must drive divided by the input capacitance presented by the path. This is more convenient than defining path electrical effort as the product of stage electrical efforts because we do not know the individual stage electrical efforts until gate sizes are selected.

$$H = \frac{C_{\text{out(path)}}}{C_{\text{in(path)}}} \tag{4.33}$$

The *path effort* F is the product of the stage efforts of each stage. Recall that the stage effort of a single stage is $f = gh$. Can we by analogy state $F = GH$ for a path?

$$F = \prod f_i = \prod g_i h_i \tag{4.34}$$

In paths that branch, $F \neq GH$. This is illustrated in Figure 4.30, a circuit with a two-way branch. Consider a path from the primary input to one of the outputs. The path logical effort is $G = 1 \times 1 = 1$. The path electrical effort is $H = 90/5 = 18$. Thus, $GH = 18$. But $F = f_1 f_2 = g_1 h_1 g_2 h_2 = 1 \times 6 \times 1 \times 6 = 36$. In other words, $F = 2GH$ in this path on account of the two-way branch.

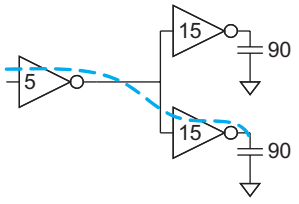


FIGURE 4.30 Circuit with two-way branch

We must introduce a new kind of effort to account for branching between stages of a path. This *branching effort* b is the ratio of the total capacitance seen by a stage to the capacitance on the path; in Figure 4.30 it is (15 + 15)/15 = 2.

$$b = \frac{C_{\text{onpath}} + C_{\text{offpath}}}{C_{\text{onpath}}} \tag{4.35}$$

The *path branching effort* B is the product of the branching efforts between stages.

$$B = \prod b_i \tag{4.36}$$

Now we can define the path effort F as the product of the logical, electrical, and branching efforts of the path. Note that the product of the electrical efforts of the stages is actually BH, not just H.

$$F = GBH \tag{4.37}$$

We can now compute the delay of a multistage network. The *path delay* D is the sum of the delays of each stage. It can also be written as the sum of the *path effort delay* D_F and *path parasitic delay* P:

$$\begin{aligned} D &= \sum d_i = D_F + P \\ D_F &= \sum f_i \\ P &= \sum p_i \end{aligned} \tag{4.38}$$

The product of the stage efforts is F, independent of gate sizes. The path effort delay is the sum of the stage efforts. The sum of a set of numbers whose product is constant is

minimized by choosing all the numbers to be equal. In other words, the path delay is minimized when each stage bears the same effort. If a path has N stages and each bears the same effort, that effort must be

$$\hat{f} = g_i h_i = F^{1/N} \tag{4.39}$$

Thus, the minimum possible delay of an N-stage path with path effort F and path parasitic delay P is

$$D = NF^{1/N} + P \tag{4.40}$$

This is a key result of Logical Effort. It shows that the minimum delay of the path can be estimated knowing only the number of stages, path effort, and parasitic delays without the need to assign transistor sizes. This is superior to simulation, in which delay depends on sizes and you never achieve certainty that the sizes selected are those that offer minimum delay.

It is also straightforward to select gate sizes to achieve this least delay. Combining EQs (4.21) and (4.22) gives us the *capacitance transformation* formula to find the best input capacitance for a gate given the output capacitance it drives.

$$C_{\text{in}_i} = \frac{C_{\text{out}_i} \times g_i}{\hat{f}} \tag{4.41}$$

Starting with the load at the end of the path, work backward applying the capacitance transformation to determine the size of each stage. Check the arithmetic by verifying that the size of the initial stage matches the specification.

Example 4.13

Estimate the minimum delay of the path from A to B in Figure 4.31 and choose transistor sizes to achieve this delay. The initial NAND2 gate may present a load of 8 λ of transistor width on the input and the output load is equivalent to 45 λ of transistor width.

SOLUTION: The path logical effort is $G = (4/3) \times (5/3) \times (5/3) = 100/27$. The path electrical effort is $H = 45/8$. The path branching effort is $B = 3 \times 2 = 6$. The path effort is $F = GBH = 125$. As there are three stages, the best stage effort is $\hat{f} = \sqrt[3]{125} = 5$. The path parasitic delay is $P = 2 + 3 + 2 = 7$. Hence, the minimum path delay is $D = 3 \times 5 + 7 = 22$ in units of τ, or 4.4 FO4 inverter delays. The gate sizes are computed with the capacitance transformation from EQ (4.41) working backward along the path: $y = 45 \times (5/3)/5 = 15$. $x = (15 + 15) \times (5/3)/5 = 10$. We verify that the initial 2-input NAND gate has the specified size of $(10 + 10 + 10) \times (4/3)/5 = 8$. The transistor sizes in Figure 4.32 are chosen to give the desired amount of input capacitance while achieving equal rise and fall delays. For example, a 2-input NOR gate should have a 4:1 P/N ratio. If the total input capacitance is 15, the pMOS width must be 12 and the nMOS width must be 3 to achieve that ratio.

We can also check that our delay was achieved. The NAND2 gate delay is $d_1 = g_1 h_1 + p_1 = (4/3) \times (10 + 10 + 10)/8 + 2 = 7$. The NAND3

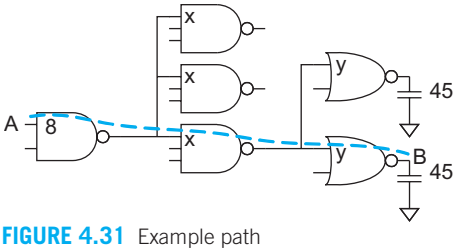


FIGURE 4.31 Example path

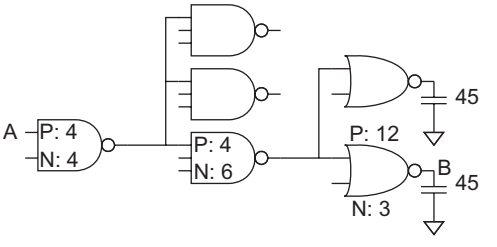


FIGURE 4.32 Example path annotated with transistor sizes

gate delay is $d_2 = g_2h_2 + p_2 = (5/3) \times (15 + 15)/10 + 3 = 8$. The NOR2 gate delay is $d_3 = g_3h_3 + p_3 = (5/3) \times 45/15 + 2 = 7$. Hence, the path delay is 22, as predicted.

Recall that delay is expressed in units of τ. In a 65 nm process with $\tau = 3$ ps, the delay is 66 ps. Alternatively, a fanout-of-4 inverter delay is 5τ, so the path delay is 4.4 FO4s.

Many inexperienced designers know that wider transistors offer more current and thus try to make circuits faster by using bigger gates. Increasing the size of any of the gates except the first one only makes the circuit slower. For example, increasing the size of the NAND3 makes the NAND3 faster but makes the NAND2 slower, resulting in a net speed loss. Increasing the size of the initial NAND2 gate does speed up the circuit under consideration. However, it presents a larger load on the path that computes input *A*, making that path slower. Hence, it is crucial to have a specification of not only the load the path must drive but also the maximum input capacitance the path may present.

4.5.2 Choosing the Best Number of Stages

Given a specific circuit topology, we now know how to estimate delay and choose gate sizes. However, there are many different topologies that implement a particular logic function. Logical Effort tells us that NANDs are better than NORs and that gates with few inputs are better than gates with many. In this section, we will also use Logical Effort to predict the best number of stages to use.

Logic designers sometimes estimate delay by counting the number of stages of logic, assuming each stage has a constant "gate delay." This is potentially misleading because it implies that the fastest circuits are those that use the fewest stages of logic. Of course, the gate delay actually depends on the electrical effort, so sometimes using fewer stages results in more delay. The following example illustrates this point.

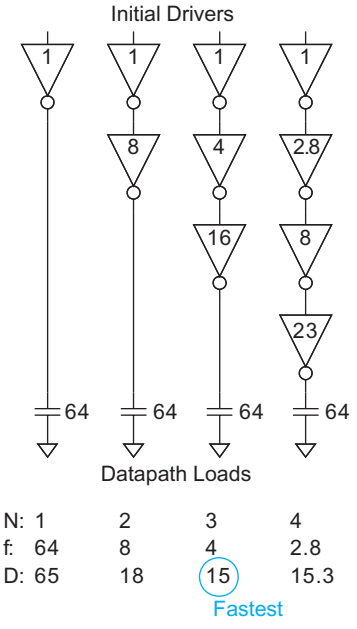


FIGURE 4.33 Comparison of different number of stages of buffers

Example 4.14

A control unit generates a signal from a unit-sized inverter. The signal must drive unit-sized loads in each bitslice of a 64-bit datapath. The designer can add inverters to buffer the signal to drive the large load. Assuming polarity of the signal does not matter, what is the best number of inverters to add and what delay can be achieved?

SOLUTION: Figure 4.33 shows the cases of adding 0, 1, 2, or 3 inverters. The path electrical effort is $H = 64$. The path logical effort is $G = 1$, independent of the number of inverters. Thus, the path effort is $F = 64$. The inverter sizes are chosen to achieve equal stage effort. The total delay is $D = N\sqrt[N]{64} + N$.

The 3-stage design is fastest and far superior to a single stage. If an even number of inversions were required, the two- or four-stage designs are promising. The four-stage design is slightly faster, but the two-stage design requires significantly less area and power.

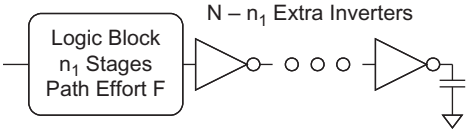


FIGURE 4.34 Logic block with additional inverters

In general, you can always add inverters to the end of a path without changing its function (save possibly for polarity). Let us compute how many should be added for least delay. The logic block shown in Figure 4.34 has n_1 stages and a path effort of F. Consider adding $N - n_1$ inverters to the end to bring the path to N stages. The extra inverters do not change the path logical effort but do add

parasitic delay. The delay of the new path is

$$D = NF^{1/N} + \sum_{i=1}^{n_1} p_i + \left(N - n_1\right) p_{\text{inv}} \tag{4.42}$$

Differentiating with respect to N and setting to 0 allows us to solve for the best number of stages, which we will call $\hat{N}$. The result can be expressed more compactly by defining

$$\rho = F^{1/\hat{N}}$$

to be the best stage effort.

$$\begin{aligned} \frac{\partial D}{\partial N} &= -F^{1/N} \ln F^{1/N} + F^{1/N} + p_{\text{inv}} = 0 \\ &\Rightarrow p_{\text{inv}} + \rho\left(1 - \ln \rho\right) = 0 \end{aligned} \tag{4.43}$$

EQ (4.43) has no closed form solution. Neglecting parasitics (i.e., assuming $p_{\text{inv}} = 0$), we find the classic result that the stage effort $\rho = 2.71828$ (e) [Mead80]. In practice, the parasitic delays mean each inverter is somewhat more costly to add. As a result, it is better to use fewer stages, or equivalently a higher stage effort than e. Solving numerically, when $p_{\text{inv}} = 1$, we find $\rho = 3.59$.

A path achieves least delay by using $\hat{N} = \log_\rho F$ stages. It is important to understand not only the best stage effort and number of stages but also the sensitivity to using a different number of stages. Figure 4.35 plots the delay increase using a particular number of stages against the total number of stages, for $p_{\text{inv}} = 1$. The x-axis plots the ratio of the actual number of stages to the ideal number. The y-axis plots the ratio of the actual delay to the best achievable. The curve is flat around the optimum. The delay is within 15% of the best achievable if the number of stages is within 2/3 to 3/2 times the theoretical best number (i.e., ρ is in the range of 2.4 to 6).

Using a stage effort of 4 is a convenient choice and simplifies mentally choosing the best number of stages. This effort gives delays within 2% of minimum for p_{inv} in the range of 0.7 to 2.5. This further explains why a fanout-of-4 inverter has a "representative" logic gate delay.

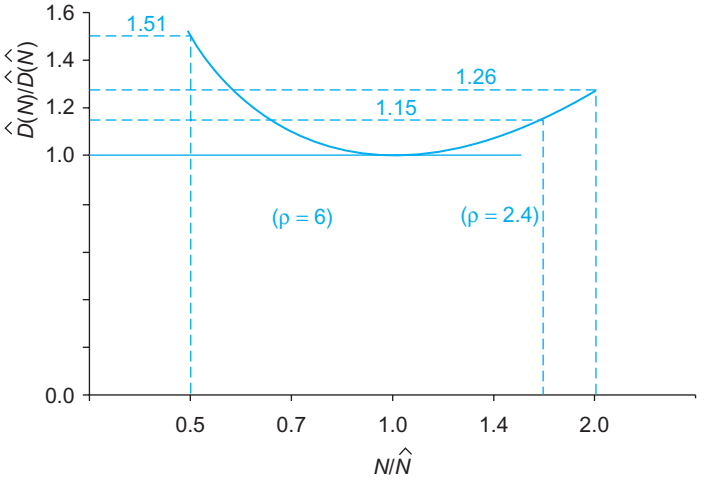


FIGURE 4.35 Sensitivity of delay to number of stages

4.5.3 Example

Consider a larger example to illustrate the application of Logical Effort. Our esteemed colleague Ben Bitdiddle is designing a decoder for a register file in the Motoroil 68W86, an embedded processor for automotive applications. The decoder has the following specifications:

- 16-word register file
- 32-bit words
- Each register bit presents a load of three unit-sized transistors on the word line (two unit-sized access transistors plus some wire capacitance)
- True and complementary versions of the address bits $A[3:0]$ are available
- Each address input can drive 10 unit-sized transistors

As we will see further in Section 12.2.2, a 2^N-word decoder consists of 2^N N-input AND gates. Therefore, the problem is reduced to designing a suitable 4-input AND gate. Let us help Ben determine how many stages to use, how large each gate should be, and how fast the decoder can operate.

The output load on a word line is 32 bits with three units of capacitance each, or 96 units. Therefore, the path electrical effort is $H = 96/10 = 9.6$. Each address is used to compute half of the 16 word lines; its complement is used for the other half. Therefore, a $B = 8$-way branch is required somewhere in the path. Now we are faced with a chicken-and-egg dilemma. We need to know the path logical effort to calculate the path effort and best number of stages. However, without knowing the best number of stages, we cannot sketch a path and determine the logical effort for that path. There are two ways to resolve the dilemma. One is to sketch a path with a random number of stages, determine the path logical effort, and then use that to compute the path effort and the actual number of stages. The path can be redesigned with this number of stages, refining the path logical effort. If the logical effort changes significantly, the process can be repeated. Alternatively, we know that the logic of a decoder is rather simple, so we can ignore the logical effort (assume $G = 1$). Then we can proceed with our design, remembering that the best number of stages is likely slightly higher than predicted because we neglected logical effort.

Taking the second approach, we estimate the path effort is $F = GBH = (1)(8)(9.6) = 76.8$. Targeting a best stage effort of $\rho = 4$, we find the best number of stages is $N = \log_4 76.8 = 3.1$. Let us select a 3-stage design, recalling that a 4-stage design might be a good choice too when logical effort is considered. Figure 4.36 shows a possible 3-stage design (INV-NAND4-INV).

The path has a logical effort of $G = 1 \times (6/3) \times 1 = 2$, so the actual path effort is $F = (2)(8)(9.6) = 154$. The stage effort is $\hat{f} = 154^{1/3} = 5.36$. This is in the reasonable range of 2.4 to 6, so we expect our design to be acceptable. Applying the capacitance transformation, we find gate sizes $z = 96 \times 1/5.36 = 18$ and $y = 18 \times 2 /5.36 = 6.7$. The delay is $3 \times 5.36 + 1 + 4 + 1 = 22.1$.

Logical Effort also allows us to rapidly compare alternative designs using a spreadsheet rather than a schematic editor and a large number of simulations. Table 4.4 compares a number of alternative designs. We find a 4-stage design is somewhat faster, as we suspected. The 4-stage NAND2-INV-NAND2-INV design not only has the theoretical best number of stages, but also uses simpler 2-input gates to reduce the logical effort and parasitic delay to obtain a 12% speedup over the original design. However, the 3-stage design has a smaller total gate area and dissipates less power.

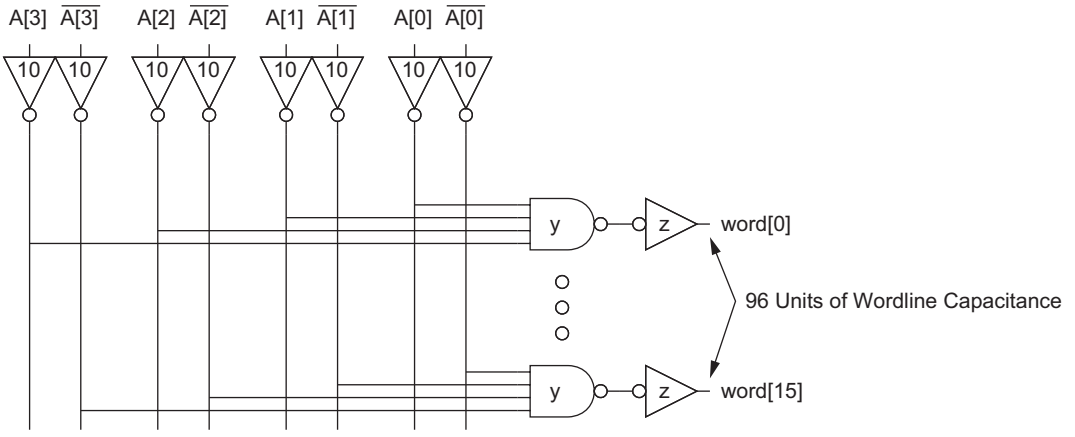


FIGURE 4.36 3-stage decoder design

TABLE 4.4 Spreadsheet comparing decoder designs

Design	Stages *N*	*G*	*P*	*D*
NAND4-INV	2	2	5	29.8
NAND2-NOR2	2	20/9	4	30.1
INV-NAND4-INV	**3**	**2**	**6**	**22.1**
NAND4-INV-INV-INV	4	2	7	21.1
NAND2-NOR2-INV-INV	4	20/9	6	20.5
NAND2-INV-NAND2-INV	4	16/9	6	19.7
INV-NAND2-INV-NAND2-INV	5	16/9	7	20.4
NAND2-INV-NAND2-INV-INV-INV	6	16/9	8	21.6

4.5.4 Summary and Observations

Logical Effort provides an easy way to compare and select circuit topologies, choose the best number of stages for a path, and estimate path delay. The notation takes some time to become natural, but this author has poured through all the letters in the English and Greek alphabets without finding better notation. It may help to remember *d* for "**d**elay," *p* for "**p**arasitic," *b* for "**b**ranching," *f* for "e**f**fort," *g* for "lo**g**ical effort" (or perhaps **g**ain), and *h* as the next letter after "f" and "g." The notation is summarized in Table 4.5 for both stages and paths.

The method of Logical Effort is applied with the following steps:

1. Compute the path effort: $F = GBH$
2. Estimate the best number of stages: $\hat{N} = \log_4 F$
3. Sketch a path using: $\hat{N}$ stages
4. Estimate the minimum delay: $D = \hat{N}F^{1/\hat{N}} + P$
5. Determine the best stage effort: $\hat{f} = F^{1/\hat{N}}$
6. Starting at the end, work backward to find sizes: $C_{\text{in}_i} = \dfrac{C_{\text{out}_i} \times g_i}{\hat{f}}$

TABLE 4.5 Summary of Logical Effort notation

Term	Stage Expression	Path Expression
number of stages	1	N
logical effort	g (see Table 4.2)	$G = \prod g_i$
electrical effort	$h = \frac{C_{out}}{C_{in}}$	$H = \frac{C_{out(path)}}{C_{in(path)}}$
branching effort	$b = \frac{C_{onpath} + C_{offpath}}{C_{onpath}}$	$B = \prod b_i$
effort	$f = gh$	$F = GBH$
effort delay	f	$D_F = \sum f_i$
parasitic delay	p (see Table 4.3)	$P = \sum p_i$
delay	$d = f + p$	$D = \sum d_i = D_F + P$

CAD tools are very fast and accurate at evaluating complex delay models, so Logical Effort should not be used as a replacement for such tools. Rather, its value arises from "quick and dirty" hand calculations and from the insights it lends to circuit design. Some of the key insights include:

- The idea of a numeric "logical effort" that characterizes the complexity of a logic gate or path allows you to compare alternative circuit topologies and show that some topologies are better than others.
- NAND structures are faster than NOR structures in static CMOS circuits.
- Paths are fastest when the effort delays of each stage are about the same and when these delays are close to four.
- Path delay is insensitive to modest deviations from the optimum. Stage efforts of 2.4–6 give designs within 15% of minimum delay. There is no need to make calculations to more than 1–2 significant figures, so many estimations can be made in your head. There is no need to choose transistor sizes exactly according to theory and there is little benefit in tweaking transistor sizes if the design is reasonable.
- Using stage efforts somewhat greater than 4 reduces area and power consumption at a slight cost in speed. Using efforts greater than 6–8 comes at a significant cost in speed.
- Using fewer stages for "less gate delays" does not make a circuit faster. Making gates larger also does not make a circuit faster; it only increases the area and power consumption.
- The delay of a well-designed path is about $\log_4 F$ fanout-of-4 (FO4) inverter delays. Each quadrupling of the load adds about one FO4 inverter delay to the path. Control signals fanning out to a 64-bit datapath therefore incur an amplification delay of about three FO4 inverters.

- The logical effort of each input of a gate increases through no fault of its own as the number of inputs grows. Considering both logical effort and parasitic delay, we find a practical limit of about four series transistors in logic gates and about four inputs to multiplexers. Beyond this fan-in, it is faster to split gates into multiple stages of skinnier gates.
- Inverters or 2-input NAND gates with low logical efforts are best for driving nodes with a large branching effort. Use small gates after the branches to minimize load on the driving gate.
- When a path forks and one leg is more critical than the others, buffer the noncritical legs to reduce the branching effort on the critical path.

4.5.5 Limitations of Logical Effort



Logical Effort is based on the linear delay model and the simple premise that making the effort delays of each stage equal minimizes path delay. This simplicity is the method's greatest strength, but also results in a number of limitations:

- Logical Effort does not account for interconnect. The effects of nonnegligible wire capacitance and RC delay will be revisited in Chapter 6. Logical Effort is most applicable to high-speed circuits with regular layouts where routing delay does not dominate. Such structures include adders, multipliers, memories, and other datapaths and arrays.
- Logical Effort explains how to design a critical path for maximum speed, but not how to design an entire circuit for minimum area or power given a fixed speed constraint. This problem is addressed in Section 5.2.2.1.
- Paths with nonuniform branching or reconvergent fanout are difficult to analyze by hand.
- The linear delay model fails to capture the effect of input slope. Fortunately, edge rates tend to be about equal in well-designed circuits with equal effort delay per stage.

4.5.6 Iterative Solutions for Sizing



To address the limitations in the previous section, we can write the delay equations for each gate in the system and minimize the latest arrival time. No closed-form solutions exist, but the equations are easy to solve iteratively on a computer and the formulation still gives some insight for the designer. This section examines sizing for minimum delay, while Section 5.2.2.1 examines sizing for minimum energy subject to a delay constraint.

The ith gate is characterized by its logical effort, g_i, parasitic delay, p_i, and drive, x_i. Formally, our goal is to find a nonnegative vector of drives x that minimizes the arrival time of the latest output. This can be done using a commercial optimizer such as MOSEK or, for smaller problems, Microsoft Excel's solver. The arrival time equations are classified as *convex*, which has the pleasant property of having a single optimum; there is no risk of finding a wrong answer. Moreover, they are of a special class of functions called *posynomials*, which allows an especially efficient technique called *geometric programming* to be applied [Fishburn85].

Example 4.15

The circuit in Figure 4.37 has nonuniform branching, reconvergent fanout, and a wire load in the middle of the path, all of which stymie back-of-the-envelope application of Logical Effort. The wire load is given in the same units as the gate capacitances (i.e., multiples of the capacitance of a unit inverter). Assume the inputs arrive at time 0. Write an expression for the arrival time of the output as a function of the gate drives. Determine the sizes to achieve minimum delay.

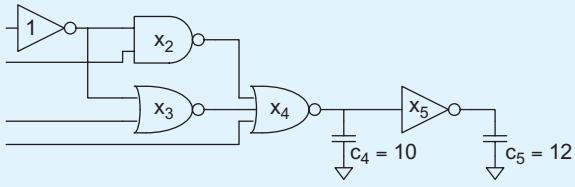


FIGURE 4.37 Example path

SOLUTION: The delay equations for each gate are obtained using EQ (4.25). Note that x indicates drive, not size. According to EQ (4.24), the input capacitance of a gate with logical effort g and drive x is $C_{in} = gx$.

$$\begin{aligned}
d_1 &= 1 + \tfrac{4}{3}x_2 + \tfrac{5}{3}x_3 \\
d_2 &= 2 + \tfrac{7}{3}\frac{x_4}{x_2} \\
d_3 &= 2 + \tfrac{7}{3}\frac{x_4}{x_3} \\
d_4 &= 3 + \frac{x_5}{x_4} + \frac{10}{x_4} \\
d_5 &= 1 + \frac{12}{x_5}
\end{aligned} \tag{4.44}$$

Write the arrival times using the definitions from EQ (4.1).

$$\begin{aligned}
a_1 &= d_1 \\
a_2 &= a_1 + d_2 \\
a_3 &= a_1 + d_3 \\
a_4 &= \max\{a_2, a_3\} + d_4 \\
a_5 &= a_4 + d_5 = d_1 + \max\{d_2, d_3\} + d_4 + d_5
\end{aligned} \tag{4.45}$$

Use a solver to choose the drives to minimize the latest arrival time. Table 4.6 summarizes the results. The minimum delay is 23.44.

The example leads to several interesting observations:

- In paths that branch, each fork should contribute equal delay. If one fork were faster than the other, it could be downsized to reduce the capacitance it presents to the stage before the branch.

- The stage efforts, f, are equal for each gate in paths with no fixed capacitive loads, but may change after a load.
- To minimize delay, upsize gates on nodes with large fixed capacitances to reduce the effort borne by the gate, while only slightly increasing the effort borne by the predecessor.

TABLE 4.6 Path design for minimum delays

Stage (i)	x_i	f_i	c_{in}	d_i	a_i
1: INV	1	4.85	1	5.85	5.85
2: NAND2	1.62	4.85	2.16	6.85	12.70
3: NOR2	1.62	4.85	2.70	6.85	12.70
4: NOR3	3.37	4.85	7.86	7.85	20.55
5: INV	6.35	1.89	6.35	2.89	23.44

A standard cell library offers a discrete set of sizes. Gate drives must be rounded to the nearest available size. For example, the circuit might use inv_1x, nand2_2x, nor2_2x, nor3_3x, and inv_6x. The delay increases to 23.83, less than a 2% penalty. In general, libraries with a granularity of $\sqrt{2}$ between successive drives are nearly as good as those with continuous sizes, so long as large inverters are available to drive big loads. Even using a granularity of 2 between drives (1x, 2x, 4x, 8x) is sufficient to obtain good results.

Although this section used a linear delay model to build on the insights of Logical Effort, it is also possible to use more elaborate models taking into account sensitivity to edge rate, V_{DD}, and V_t [Patil07]; the extra complexity is not a problem for a numerical solver and the model allows for optimizing supply and threshold voltages as well as sizes. Timing models are discussed further in Section 4.6.

4.6 Timing Analysis Delay Models



To handle a chip with millions of gates, the delay model for a timing analyzer must be easy enough to compute that timing analysis is fast, yet accurate enough to give confidence. This section reviews several delay models for timing analysis that are much faster than SPICE simulations, yet more accurate than the simple linear delay model. Timing (and area, power, and noise) models for each gate in a standard cell library are stored in a .lib file. These models are part of the Liberty standard documented at `www.opensourceliberty.org`. Logical effort parameters for standard cells can be obtained by fitting a straight line to the timing models, assuming equal delays and rise/fall times for the previous stage.

4.6.1 Slope-Based Linear Model

A simple approach is to extend the linear delay model by adding a term reflecting the input slope. Assuming the slope of the input is proportional to the delay of the previous stage, the delays for rising and falling outputs can be expressed as:

```
delay_rise =intrinsic_rise + rise_resistance × capacitance +
            slope_rise × delay_previous
delay_fall =intrinsic_fall + fall_resistance × capacitance +
            slope_fall × delay_previous
```

Linear delay models are not accurate enough to handle the wide range of slopes and loads found in synthesized circuits, so they have largely been superseded by nonlinear delay models.

4.6.2 Nonlinear Delay Model

A nonlinear delay model looks up the delay from a table based on the load capacitance and the input slope. Separate tables are used to lookup rising and falling delays and output slopes. Table 4.7 shows an example of a nonlinear delay model for the falling delay of an inverter. The timing analyzer uses interpolation when a specific load capacitance or slope is not in the table.

TABLE 4.7 Nonlinear Delay Model for inverter t_{pdf} (ps)

C_{out} (fF)	Rise Time (ps)				
	10	20	40	80	160
1	11.5	13.3	17.0	21.2	25.3
2	18.4	20.2	24.1	30.9	38.5
4	32.0	33.8	37.6	43.4	58.5
8	59.2	60.9	65.7	72.3	87.8

Nonlinear delay models are widely used at the time of this writing. However, they do not contain enough information to characterize the delay of a gate driving a complex RC interconnect network with the accuracy desired by some users. They also lack the accuracy to fully characterize noise events. A different model must be created for each voltage and temperature at which the chip might be characterized.

4.6.3 Current Source Model

The limitations of nonlinear delay models have motivated the development of current source models. A *current source model* theoretically should express the output DC current as a nonlinear function of the input and output voltages of the cell. A timing analyzer numerically integrates the output current to find the voltage as a function of time into an arbitrary RC network and to solve for the propagation delay.

The Liberty *Composite Current Source Model* (CCSM) instead stores output current as a function of time for a given input slew rate and output capacitance. The competing *Effective Current Source Model* (ECSM) stores output voltage as a function of time. The two representations are equivalent, and can be synthesized into a true current source model [Chopra06].

4.7 Pitfalls and Fallacies

Defining gate delay for an unloaded gate

When marketing a process, it is common to report gate delay based on an inverter in a ring oscillator (2τ), or even the RC time constant of a transistor charging its own gate capacitance ($1/3\ \tau$). Remember that the delay of a real gate on the critical path should be closer to $5–6\tau$.

When in doubt, ask how "gate delay" is defined or ask for the FO4 inverter delay.

Trying to increase speed by increasing the size of transistors in a path

Most designers know that increasing the size of a transistor decreases its resistance and thus makes it faster at driving a constant load. Novice designers sometimes forget that increasing the size increases input capacitance and makes the previous stage slower, especially when that previous stage belongs to somebody else's timing budget. The authors have seen this lead to lack of convergence in full-chip timing analysis on a large microprocessor because individual engineers boost the size of their own gates until their path meets timing. Only after the weekly full-chip timing roll-up do they discover that their inputs now arrive later because of the greater load on the previous stage. The solution is to include in the specification of each block not only the arrival time but also the resistance of the driver in the previous block.

Trying to increase speed by using as few stages of logic as possible

Logic designers often count "gate delays" in a path. This is a convenient simplification when used properly. In the hands of an inexperienced engineer who believes each gate contributes a gate delay, it suggests that the delay of a path is minimized by using as few stages of logic as possible, which is clearly untrue.

4.8 Historical Perspective

Figure 1.5 illustrated the exponential increase in microprocessor frequencies over nearly four decades. While much of the improvement comes from the natural improvements in gate delay with feature size, a significant portion is due to better microarchitecture and circuit design with fewer gate delays per cycle. From a circuit perspective, the cycle time is best expressed in FO4 inverter delays.

Figure 4.38 illustrates the historical trends in microprocessor cycle time based on chips reported at the International Solid-State Circuits Conference. Early processors operated at close to 100 FO4 delays per cycle. The Alpha line of microprocessors from Digital Equipment Corporation shocked the staid world of circuit design in the early 1990s by proving that cycle times below 20 FO4 delays were possible. This kicked off a race for higher clock frequencies. By the late 1990s, Intel and AMD marketed processors primarily on frequency. The Pentium II and III reached about 20–24 FO4 delays/cycle. The Pentium 4 drove cycle times down to about 10 FO4 at the expense of a very long

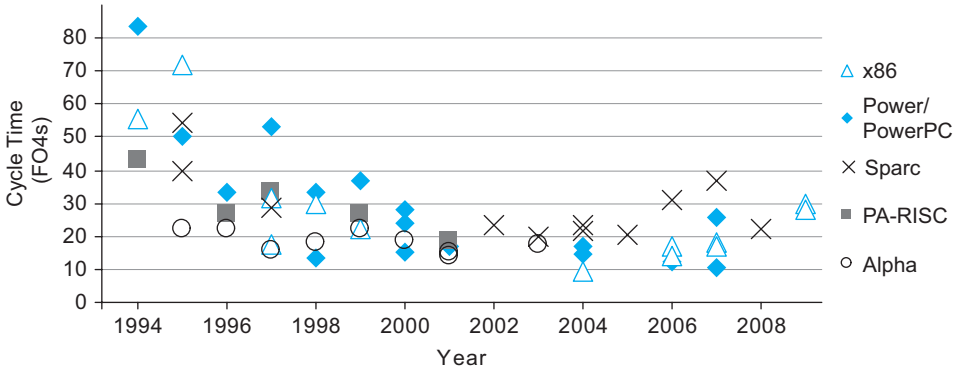


FIGURE 4.38 Microprocessor cylcle time trends. Data has some uncertainty based on estimating FO4 delay as a function of feature size.

pipeline and enormous power consumption. Microarchitects predicted that performance would be maximized at a cycle time of only 8 FO4 delays/cycle [Hrishikesh02].

The short cycle times came at the expense of vast numbers (20–30) of pipeline stages and enormous power consumption (nearly 100 W). As will be seen in the next chapter, power became as important as performance specifications. The number of gates per cycle rebounded to a more power-efficient point. [Srinivasan02] observed that 19–24 FO4 delays per cycle provides a better trade-off between performance and power.

Application-specific integrated circuits have generally operated at much lower frequencies (e.g., 200–400 MHz in nanometer processes) so that they can be designed more easily. Typical ASIC cycle times are 40–100 FO4 delays per cycle [Mai05, Chinnery02], although performance-critical designs sometimes are as fast as 25 FO4s.

Summary

The VLSI designer's challenge is to engineer a system that meets speed requirements while consuming little power or area, operating reliably, and taking little time to design. Circuit simulation is an important tool for calculating delay and will be discussed in depth in Chapter 5, but it takes too long to simulate every possible design; is prone to garbage-in, garbage-out mistakes; and doesn't give insight into why a circuit has a particular delay or how the circuit should be changed to improve delay. The designer must also have simple models to quickly estimate performance by hand and explain why some circuits are better than others.

Although transistors are complicated devices with nonlinear current-voltage and capacitance-voltage relationships, for the purpose of delay estimation in digital circuits, they can be approximated quite well as having constant capacitance and an effective resistance R when ON. Logic gates are thus modeled as RC networks. The Elmore delay model estimates the delay of the network as the sum of each capacitance times the resistance through which it must be charged or discharged. Therefore, the gate delay consists of a parasitic delay (accounting for the gate driving its own internal parasitic capacitance) plus an effort delay (accounting for the gate driving an external load). The effort delay depends on the electrical effort (the ratio of load capacitance to input capacitance, also called *fanout*) and the logical effort (which characterizes the current driving capability of the gate relative to an inverter with equal input capacitance). Even in advanced fabrication processes, the delay vs. electrical effort curve fits a straight line very well. The method of Logical Effort builds on this linear delay model to help us quickly estimate the delay of entire paths based on the effort and parasitic delay of the path. We will use Logical Effort in subsequent chapters to explain what makes circuits fast.

Exercises

4.1 Sketch a 2-input NOR gate with transistor widths chosen to achieve effective rise and fall resistances equal to a unit inverter. Compute the rising and falling propagation delays of the NOR gate driving h identical NOR gates using the Elmore delay model. Assume that every source or drain has fully contacted diffusion when making your estimate of capacitance.

4.2 Sketch a stick diagram for the 2-input NOR. Repeat Exercise 4.1 with better capacitance estimates. In particular, if a diffusion node is shared between two parallel

transistors, only budget its capacitance once. If a diffusion node is between two series transistors and requires no contacts, only budget half the capacitance because of the smaller diffusion area.

4.3 Find the rising and falling propagation delays of an unloaded AND-OR-INVERT gate using the Elmore delay model. Estimate the diffusion capacitance based on a stick diagram of the layout.

4.4 Find the worst-case Elmore parasitic delay of an n-input NOR gate.

4.5 Sketch a delay vs. electrical effort graph like that of Figure 4.21 for a 2-input NOR gate using the logical effort and parasitic delay estimated in Section 4.4.2. How does the slope of your graph compare to that of a 2-input NAND? How does the y-intercept compare?

4.6 Let a 4x inverter have transistors four times as wide as those of a unit inverter. If a unit inverter has three units of input capacitance and parasitic delay of p_{inv}, what is the input capacitance of a 4x inverter? What is the logical effort? What is the parasitic delay?

4.7 A 3-stage logic path is designed so that the effort borne by each stage is 12, 6, and 9 delay units, respectively. Can this design be improved? Why? What is the best number of stages for this path? What changes do you recommend to the existing design?

4.8 Suppose a unit inverter with three units of input capacitance has unit drive.

a) What is the drive of a 4x inverter?

b) What is the drive of a 2-input NAND gate with three units of input capacitance?

4.9 Sketch a 4-input NAND gate with transistor widths chosen to achieve equal rise and fall resistance as a unit inverter. Show why the logical effort is 6/3.

4.10 Consider the two designs of a 2-input AND gate shown in Figure 4.39. Give an intuitive argument about which will be faster. Back up your argument with a calculation of the path effort, delay, and input capacitances x and y to achieve this delay.

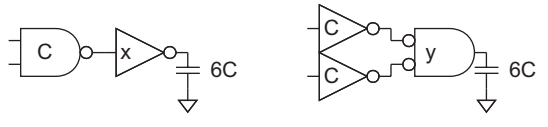


FIGURE 4.39 2-input AND gate

4.11 Consider four designs of a 6-input AND gate shown in Figure 4.40. Develop an expression for the delay of each path if the path electrical effort is H. What design is fastest for $H = 1$? For $H = 5$? For $H = 20$? Explain your conclusions intuitively.

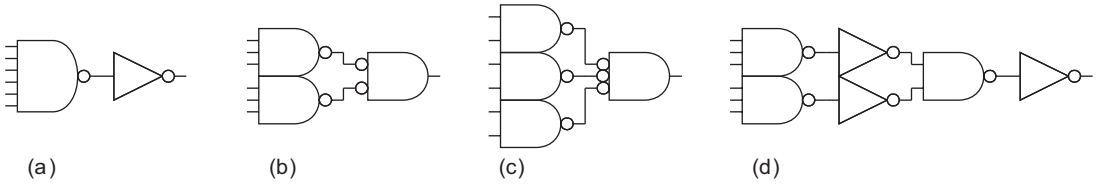


FIGURE 4.40 6-input AND gate

4.12 Repeat the decoder design example from Section 4.5.3 for a 32-word register file with 64-bit registers. Determine the fastest decoder design and estimate the delay of the decoder and the transistor widths to achieve this delay.

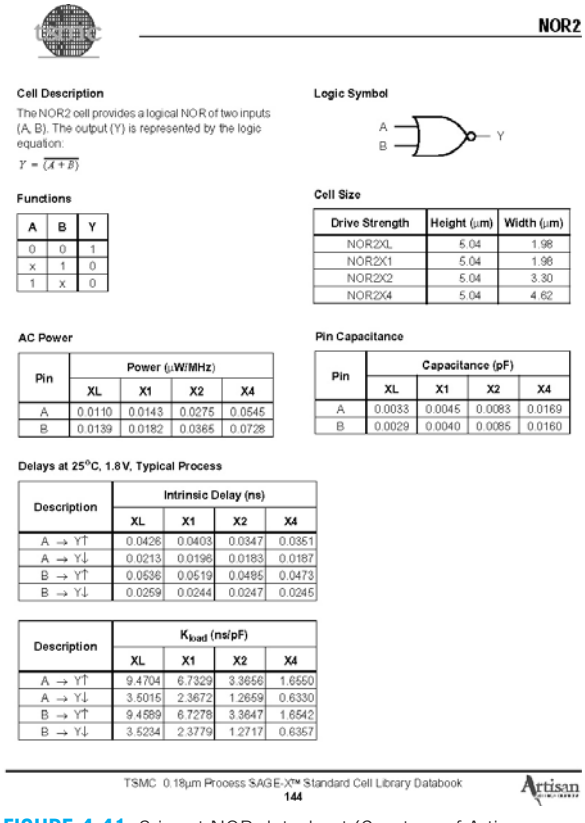
4.13 Design a circuit at the gate level to compute the following function:

```
if (a == b) y = a;
else y = 0;
```

Let *a*, *b*, and *y* be 16-bit busses. Assume the input and output capacitances are each 10 units. Your goal is to make the circuit as fast as possible. Estimate the delay in FO4 inverter delays using Logical Effort if the best gate sizes were used. What sizes do you need to use to achieve this delay?

4.14 Plot the average delay from input *A* of an FO3 NAND2 gate from the datasheet in Figure 4.25. Why is the delay larger for the XL drive strength than for the other drive strengths?

4.15 Figure 4.41 shows a datasheet for a 2-input NOR gate in the Artisan Components standard cell library for the TSMC 180 nm process. Find the average parasitic delay and logical effort of the X1 NOR gate *A* input. Use the value of τ from Section 4.4.5.



NOR2

Cell Description

The NOR2 cell provides a logical NOR of two inputs (A, B). The output (Y) is represented by the logic equation:

$Y = \overline{(A + B)}$

Logic Symbol

Functions

A	B	Y
0	0	1
x	1	0
1	x	0

Cell Size

Drive Strength	Height (μm)	Width (μm)
NOR2XL	5.04	1.98
NOR2X1	5.04	1.98
NOR2X2	5.04	3.30
NOR2X4	5.04	4.62

AC Power

Pin	Power (μW/MHz)			
	XL	X1	X2	X4
A	0.0110	0.0143	0.0275	0.0545
B	0.0139	0.0182	0.0365	0.0728

Pin Capacitance

Pin	Capacitance (pF)			
	XL	X1	X2	X4
A	0.0033	0.0045	0.0083	0.0169
B	0.0029	0.0040	0.0085	0.0160

Delays at 25°C, 1.8V, Typical Process

Description	Intrinsic Delay (ns)			
	XL	X1	X2	X4
A → Y↑	0.0426	0.0403	0.0347	0.0351
A → Y↓	0.0213	0.0196	0.0183	0.0187
B → Y↑	0.0536	0.0519	0.0495	0.0473
B → Y↓	0.0259	0.0244	0.0247	0.0245

Description	K_{load} (ns/pF)			
	XL	X1	X2	X4
A → Y↑	9.4704	6.7329	3.3656	1.6550
A → Y↓	3.5015	2.3672	1.2659	0.6330
B → Y↑	9.4589	6.7278	3.3647	1.6542
B → Y↓	3.5234	2.3779	1.2717	0.6357

FIGURE 4.41 2-input NOR datasheet (Courtesy of Artisan Components.)

4.16 Find the parasitic delay and logical effort of the X2 and X4 NOR gate *A* input using Figure 4.41. By what percentage do they differ from that of the X1 gate? What does this imply about our model that parasitic delay and logical effort depend only on gate type and not on transistor sizes?

4.17 What are the parasitic delay and logical effort of the X1 NOR gate *B* input in Figure 4.41? How and why do they differ from the *A* input?

4.18 Parasitic delay estimates in Section 4.4.2 are made assuming contacted diffusion on each transistor on the output node and ignoring internal diffusion. Would parasitic delay increase or decrease if you took into account that some parallel transistors on the output node share a single diffusion contact? If you counted internal diffusion capacitance between series transistors? If you counted wire capacitance within the cell?

4.19 Consider a process in which pMOS transistors have three times the effective resistance as nMOS transistors. A unit inverter with equal rising and falling delays in this process is shown in Figure 4.42. Calculate the logical efforts of a 2-input NAND gate and a 2-input NOR gate if they are designed with equal rising and falling delays.

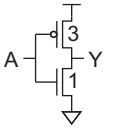


FIGURE 4.42 Unit inverter

4.20 Generalize Exercise 4.19 if the pMOS transistors have μ times the effective resistance of nMOS transistors. Find a general expression for the logical efforts of a k-input NAND gate and a k-input NOR gate. As μ increases, comment on the relative desirability of NANDs vs. NORs.

4.21 Some designers define a "gate delay" to be a fanout-of-3 2-input NAND gate rather than a fanout-of-4 inverter. Using Logical Effort, estimate the delay of a fanout-of-3 2-input NAND gate. Express your result both in τ and in FO4 inverter delays, assuming $p_{inv} = 1$.

4.22 Repeat Exercise 4.21 in a process with a lower ratio of diffusion to gate capacitance in which $p_{inv} = 0.75$. By what percentage does this change the NAND gate delay, as measured in FO4 inverter delays? What if $p_{inv} = 1.25$?

4.23 The 64-bit Naffziger adder [Naffziger96] has a delay of 930 ps in a fast 0.5-μm Hewlett-Packard process with an FO4 inverter delay of about 140 ps. Estimate its delay in a 65 nm process with an FO4 inverter delay of 20 ps.

4.24 An output pad contains a chain of successively larger inverters to drive the (relatively) enormous off-chip capacitance. If the first inverter in the chain has an input capacitance of 20 fF and the off-chip load is 10 pF, how many inverters should be used to drive the load with least delay? Estimate this delay, expressed in FO4 inverter delays.

4.25 The clock buffer in Figure 4.43 can present a maximum input capacitance of 100 fF. Both true and complementary outputs must drive loads of 300 fF. Compute the input capacitance of each inverter to minimize the worst-case delay from input to either output. What is this delay, in τ? Assume the inverter parasitic delay is 1.

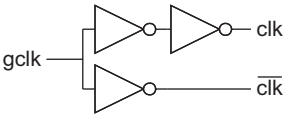


FIGURE 4.43 Clock buffer

4.26 The clock buffer from Exercise 4.25 is an example of a *1–2 fork*. In general, if a 1–2 fork has a maximum input capacitance of C_1 and each of the two legs drives a load of C_2, what should the capacitance of each inverter be and how fast will the circuit operate? Express your answer in terms of p_{inv}.

This page intentionally left blank